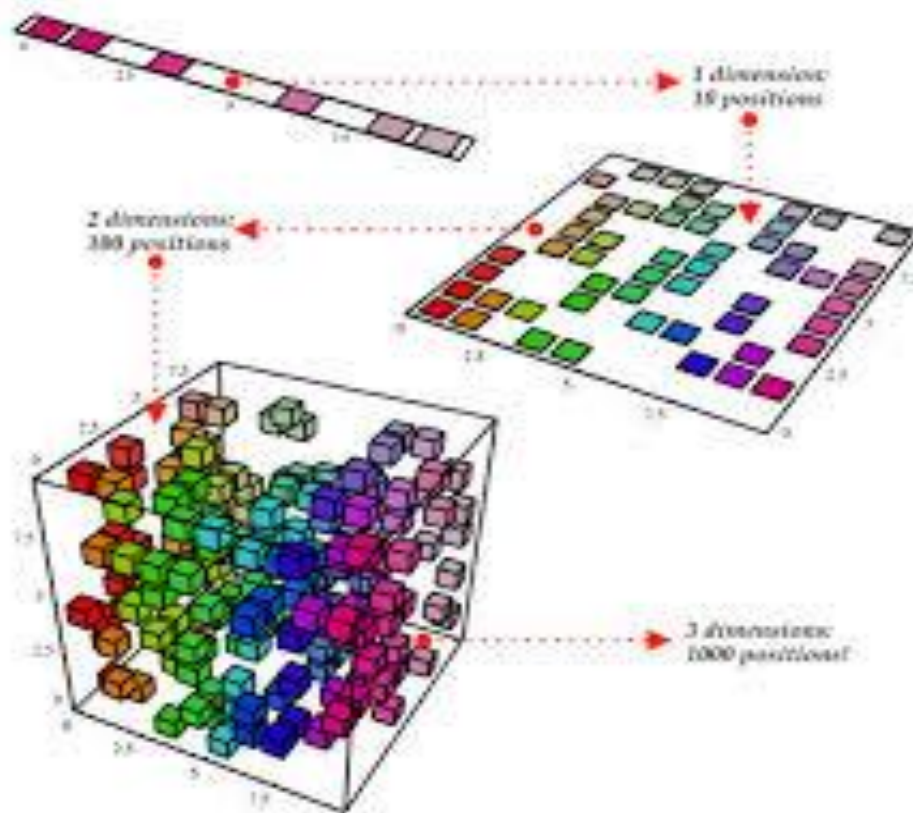


UNIT-IV DIMENSIONALITY REDUCTION



UNIT-IV: DIMENSIONALITY REDUCTION AND EVOLUTIONARY MODELS

- Dimensionality Reduction(Book-1) – Linear Discriminant Analysis – Principal Component Analysis – Factor Analysis – Independent Component Analysis – Locally Linear Embedding – Isomap – Evolutionary Learning – Genetic algorithms – Genetic Offspring: - Genetic Operators – Using Genetic Algorithms – Reinforcement Learning – Overview – Getting Lost Example

Dimensionality Reduction

- Looking at data and plotting results, never go beyond three dimensions in data, and usually find two dimensions easier to interpret.
- It reduces computational cost of many algorithms.
- Dimensionality reduction is useful to remove noise, to significantly improve the results of the learning algorithm, make the dataset easier to work with, and make the results easier to understand.
- There are three different ways to do dimensionality reduction.
- The first is feature selection, means looking through the features that are available and seeing whether or not they are actually useful, i.e., correlated to the output variables.

- Many use neural networks precisely because they don't want to 'get their hands dirty' and look at the data themselves, the results will be better if you check for correlations.
- The second method is feature derivation, which means deriving new features from the old ones, by applying transforms to the dataset that simply change the axes (coordinate system) of the graph by moving and rotating them (i.e. in terms of matrix)
- The reason for dimensionality reduction is that it combines features, and identify which are useful and which are not.
- The third method is simply to use clustering in order to group together similar data points, and to see whether this allows fewer features to be used.

- Once worked out how to represent the data, then suppress dimensions that aren't useful to the algorithm.
- Perform feature selection which is a search Problem, looking at the possible inputs that we have for the problem, and deciding which are useful.
- Take the best system so far, and then search over the set of possible next features to add.
- It is computationally very expensive, since for d features there are $2^d - 1$ possible sets of features to search over, from any individual feature up to the full set.
- In general, greedy methods are employed, although backtracking can also be employed to check whether the search gets stuck.
- Considering a method of dimensionality reduction that is aimed at supervised learning, Linear Discriminant Analysis.

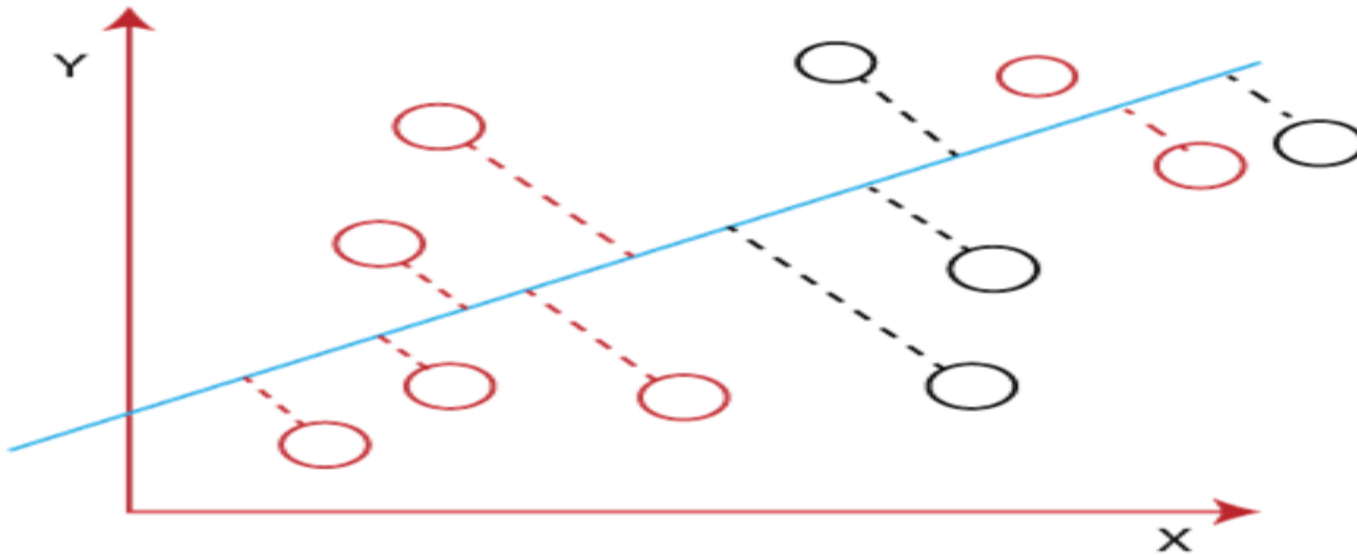
LINEAR DISCRIMINANT ANALYSIS (LDA)

- *It is also known as Normal Discriminant Analysis (NDA) or Discriminant Function Analysis (DFA).*
- *The logistic regression algorithm is limited to only two-class, linear Discriminant analysis is applicable for more than two classes of classification problems.*
- *It is a supervised classification problems in machine learning.*
- LDA is used to project the features in higher dimensional space into lower dimensional space
- LDA can also be used in data pre-processing to reduce the number of features, just as PCA, which reduces the computing cost

LINEAR DISCRIMINANT ANALYSIS (LDA)

Two criterias need to follow in LDA to create new axis:

- It maximizes the distance between means of two classes.
- It minimizes the variance within the individual class.



LINEAR DISCRIMINANT ANALYSIS (LDA)

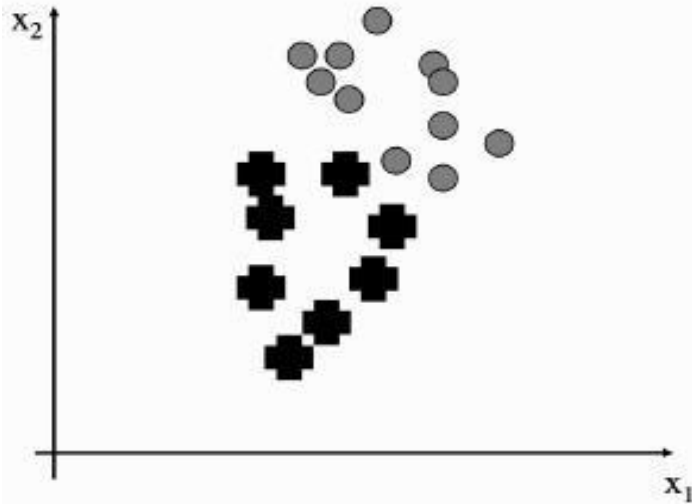


FIGURE 6.2 A set of data points in two dimensions, with two classes.

- Fig 6.2 shows a simple two-dimensional dataset consisting of two classes.
 1. Compute various statistics about the data, but we will settle for the means of the two classes in the data, μ_1 and μ_2 , the mean of the entire dataset (μ)
 2. The covariance of each class with itself.

$$\mu_1 = \frac{1}{N_1} \sum_{x \in \omega_1} x$$

$$s1 = \sum_j (x_j - \mu)(x_j - \mu)^T.$$

3. Compute the within class- scatter matrix (The principal insight of LDA is that the covariance matrix can tell us about the scatter within a dataset, which is the amount of spread that there is within the data.)

$$S_w = S_1 + S_2$$

4. Compute the between class- scatter matrix (The way to find this scatter is to multiply the covariance by the p_c , *the probability of the class (that is, the number of data points there are in that class divided by the total number)*)

$$S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

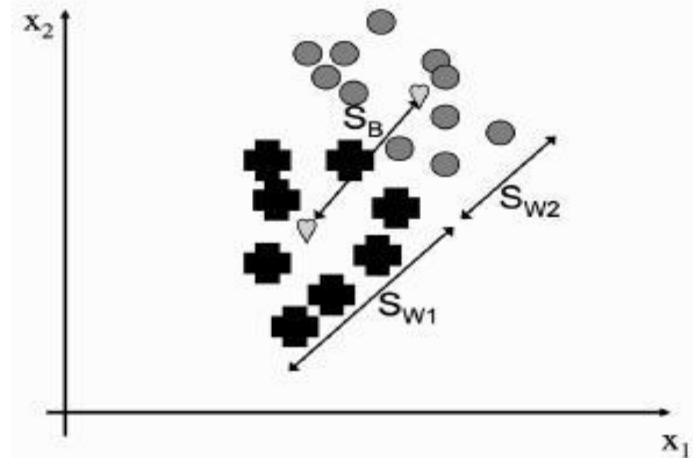
- Adding the values of this for all of the classes gives us a measure of the within-class scatter

$$S_W = \sum_{\text{classes } c} \sum_{j \in c} p_c (\mathbf{x}_j - \mu_c)(\mathbf{x}_j - \mu_c)^T. \quad (6.1)$$

- If our dataset is easy to separate into classes, then this within-class scatter should be small, so that each class is tightly clustered together and able to separate the data, the distance *between the classes to be large*.

$$S_B = \sum_{\text{classes } c} (\mu_c - \mu)(\mu_c - \mu)^T. \quad (6.2)$$

FIGURE 6.3 The meaning of the between-class and within-class scatter. The hearts mark the means of the two classes.



- The meanings of these two measurements is shown in Figure 6.3.
- The argument about good separation suggests that datasets that are easy to separate into the different classes (i.e., the classes are discriminable) should have S_B/S_W as large as possible (i.e. *Dimensionality Reduction*).
- Figure 6.4 shows two projections of the dataset onto a straight line.
- For the projection on the left it is clear that we can't separate out the two classes, while for the one on the right we can.
- find a way to compute a suitable projection.

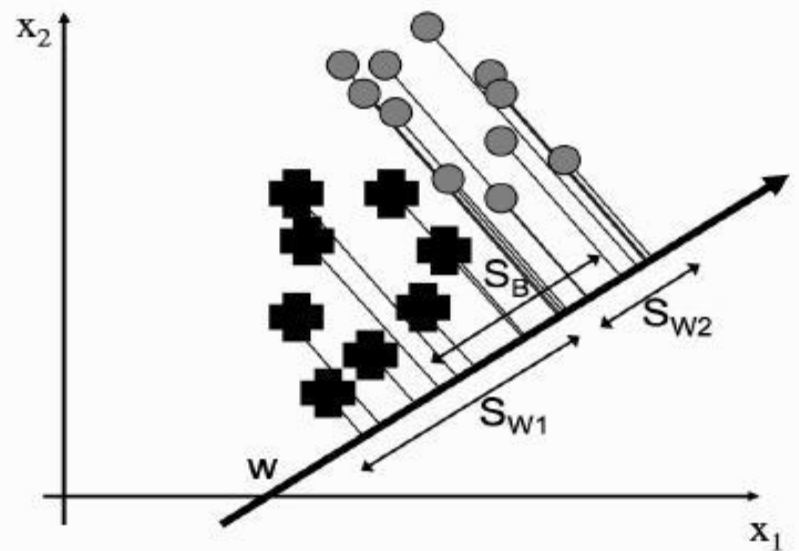
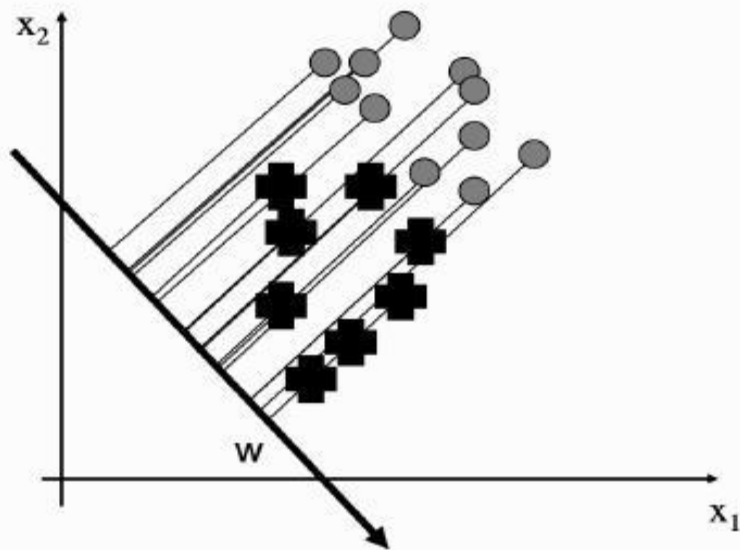


FIGURE 6.4 Two different possible projection lines. The one on the left fails to separate the classes.

- Any line can be written as a vector $\mathbf{w}$.
- The projection of the data can be written as $\mathbf{z} = \mathbf{w}^T \cdot \mathbf{x}$ for data point $\mathbf{x}$.
- This gives us a scalar that is the distance along the $\mathbf{w}$ vector that we need to go to find the projection of point $\mathbf{x}$.
- $\mathbf{w}^T \cdot \mathbf{x}$ is the sum of the vectors multiplied together element-wise, and is equal to the size of $\mathbf{w}$ times the size of $\mathbf{x}$ times the **cosine** of the **angle** between them.

- compute the projection of our data along $\mathbf{w}$ for every point, and we will have projected our data onto a straight line, as is shown in the two A^{TT} examples in Figure 6.4. μ_c^1

5) Compute Eigen values and Eigen vectors from the within class and between class scatter matrix

$$S_W^{-1} S_B \mathbf{w} = \lambda \mathbf{w}$$

6) Sort the values of eigen values and select the top k values

7) Find the Eigen vectors corresponding to the top k eigen vectors

$$\left(S_W^{-1} S_B - \lambda I \right) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = 0$$

8) Obtain LDA by taking the dot product of eigen vectors and original data.

- Since the mean can be treated as a data point, we can project

- Ignore the ratio of within-class and between-class scatter, since it is a scalar and therefore does not affect the direction of the vector.
- Unfortunately, this does not work for the general case.
- There, finding the minimum S_W^{-1} is not simple, and S_W^{-1} requires computing the generalized eigenvectors of S_B assuming that S_W^{-1} exists.
- Compute the between class and within-class scatters, and then find the value of $\mathbf{w}$.
- Figure 6.5 shows a plot of the first two dimensions of the iris data (with the classes shown as three different symbols) before and after applying LDA, with the number of dimensions being set to two.

Ex:

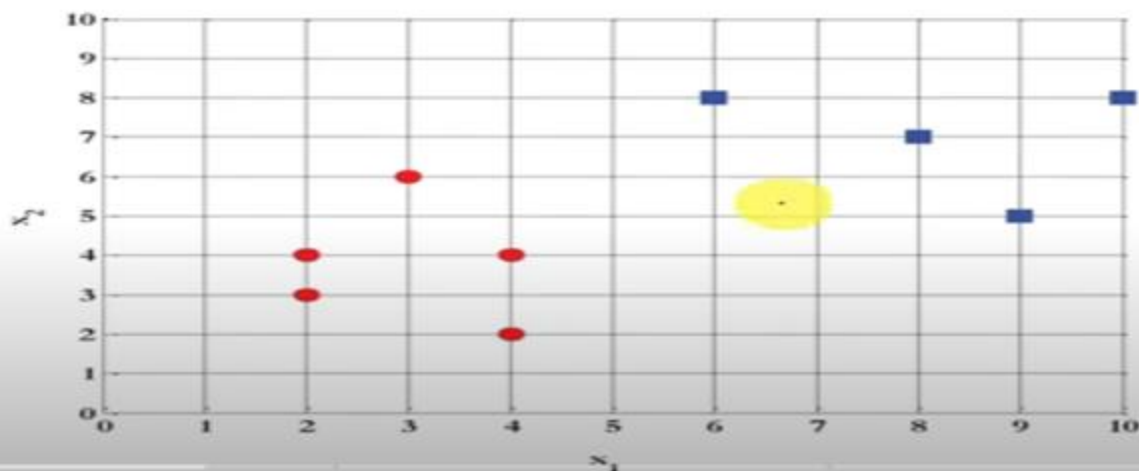
- Compute the Linear Discriminant projection for the following two dimensional dataset.

Samples for class ω_1 : $\mathbf{X}_1 = (x_1, x_2) = \{(4,2), (2,4), (2,3), (3,6), (4,4)\}$

Sample for class ω_2 : $\mathbf{X}_2 = (x_1, x_2) = \{(9,10), (6,8), (9,5), (8,7), (10,8)\}$

Samples for class ω_1 : $\mathbf{X}_1 = (x_1, x_2) = \{(4,2), (2,4), (2,3), (3,6), (4,4)\}$

Sample for class ω_2 : $\mathbf{X}_2 = (x_1, x_2) = \{(9,10), (6,8), (9,5), (8,7), (10,8)\}$



The classes mean are :

$$\mu_1 = \frac{1}{N_1} \sum_{x \in \omega_1} x = \frac{1}{5} \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 3 \\ 6 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \end{pmatrix} \right] = \begin{pmatrix} 3 \\ 3.8 \end{pmatrix}$$

$$\mu_2 = \frac{1}{N_2} \sum_{x \in \omega_2} x = \frac{1}{5} \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} + \begin{pmatrix} 6 \\ 8 \end{pmatrix} + \begin{pmatrix} 9 \\ 5 \end{pmatrix} + \begin{pmatrix} 8 \\ 7 \end{pmatrix} + \begin{pmatrix} 10 \\ 8 \end{pmatrix} \right] = \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}$$

Covariance matrix of the first class:

$$S_1 = \sum_{x \in \omega_1} \frac{(x - \mu_1)(x - \mu_1)^T}{N-1} = \frac{\left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 2 \\ 3 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 2 \\ 3 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 3 \\ 6 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 3 \\ 6 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 4 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 4 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T}{N-1}$$

$$= \begin{pmatrix} 1 & -0.25 \\ -0.25 & 2.2 \end{pmatrix}$$

Covariance matrix of the second class:

$$\begin{aligned}
 S_2 &= \sum_{x \in \theta_2} \frac{(x - \mu_2)(x - \mu_2)^T}{N-1} = \frac{1}{N-1} \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T + \begin{pmatrix} 6 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \left[\begin{pmatrix} 6 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \\
 &\quad + \begin{pmatrix} 9 \\ 5 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \left[\begin{pmatrix} 9 \\ 5 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T + \begin{pmatrix} 8 \\ 7 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \left[\begin{pmatrix} 8 \\ 7 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T + \begin{pmatrix} 10 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \left[\begin{pmatrix} 10 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \Bigg] \\
 &= \begin{pmatrix} 2.3 & -0.05 \\ -0.05 & 3.3 \end{pmatrix}
 \end{aligned}$$

Within-class scatter matrix:

$$\begin{aligned}
 S_w &= S_1 + S_2 = \begin{pmatrix} 1 & -0.25 \\ -0.25 & 2.2 \end{pmatrix} + \begin{pmatrix} 2.3 & -0.05 \\ -0.05 & 3.3 \end{pmatrix} \\
 &= \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}
 \end{aligned}$$

$$\begin{aligned} S_B &= (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T \\ &= \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \\ &= \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix} (-5.4 \quad -3.8) \\ &= \begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} \end{aligned}$$

Find Eigen Values

$$S_W^{-1}S_B w = \lambda w$$

$$\Rightarrow |S_W^{-1}S_B - \lambda I| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}^{-1} \begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 0.3045 & 0.0166 \\ 0.0166 & 0.1827 \end{pmatrix} \begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 9.2213 - \lambda & 6.489 \\ 4.2339 & 2.9794 - \lambda \end{pmatrix} \right|$$

$$= (9.2213 - \lambda)(2.9794 - \lambda) - 6.489 \times 4.2339 = 0$$

$$\Rightarrow \lambda^2 - 12.2007\lambda = 0 \Rightarrow \lambda(\lambda - 12.2007) = 0$$

$$\Rightarrow \lambda_1 = 0, \lambda_2 = 12.2007$$

Find Eigen Vector



$$\left(S_W^{-1} S_B - \lambda I \right) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = 0$$

$$w_1 = \begin{pmatrix} -0.5755 \\ 0.8178 \end{pmatrix}$$

$$w_2 = \begin{pmatrix} 0.9088 \\ 0.4173 \end{pmatrix} = w^*$$

- Obtain the LDA by taking the dot product of eigen vectors and original data

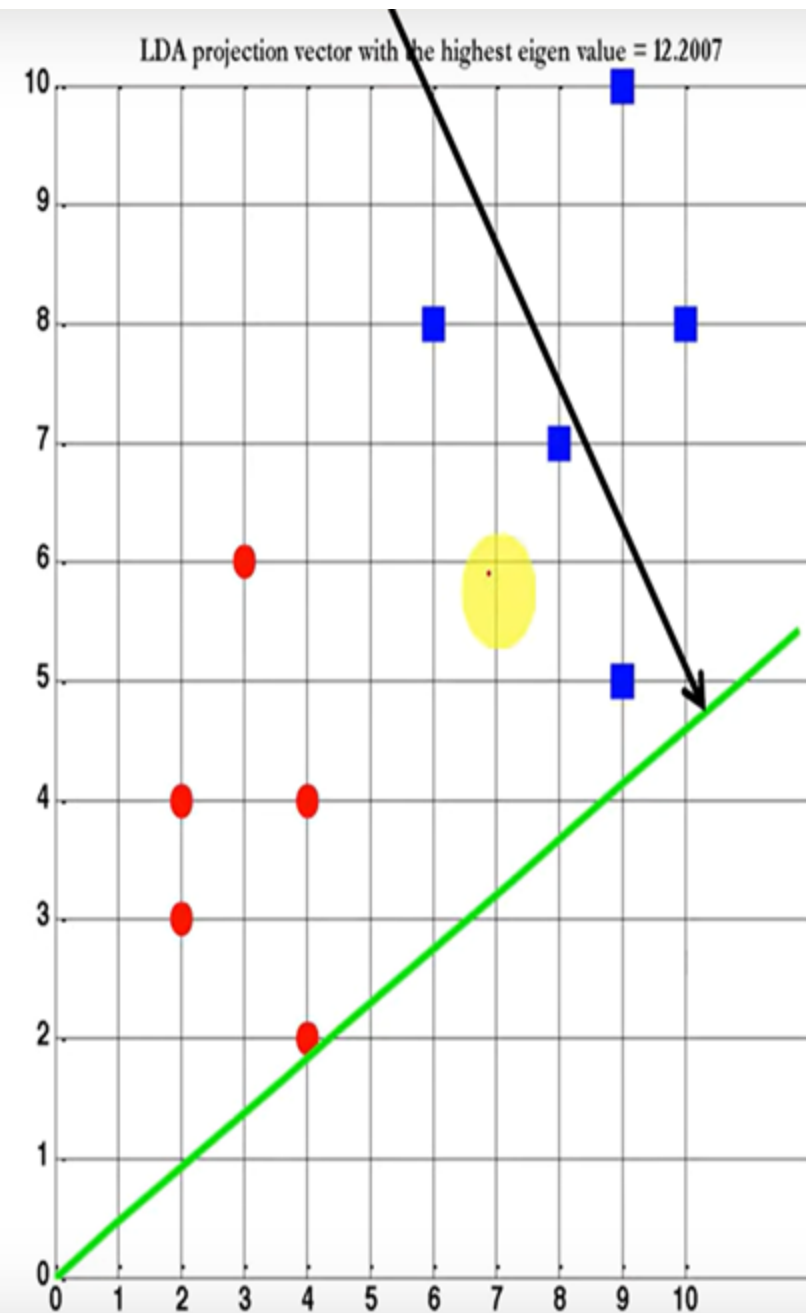
$$w_2 = \begin{pmatrix} 0.9088 \\ 0.4173 \end{pmatrix} = w^*$$

Samples for class ω_1 : $\mathbf{X}_1 = (x_1, x_2) = \{(4,2), (2,4), (2,3), (3,6), (4,4)\}$

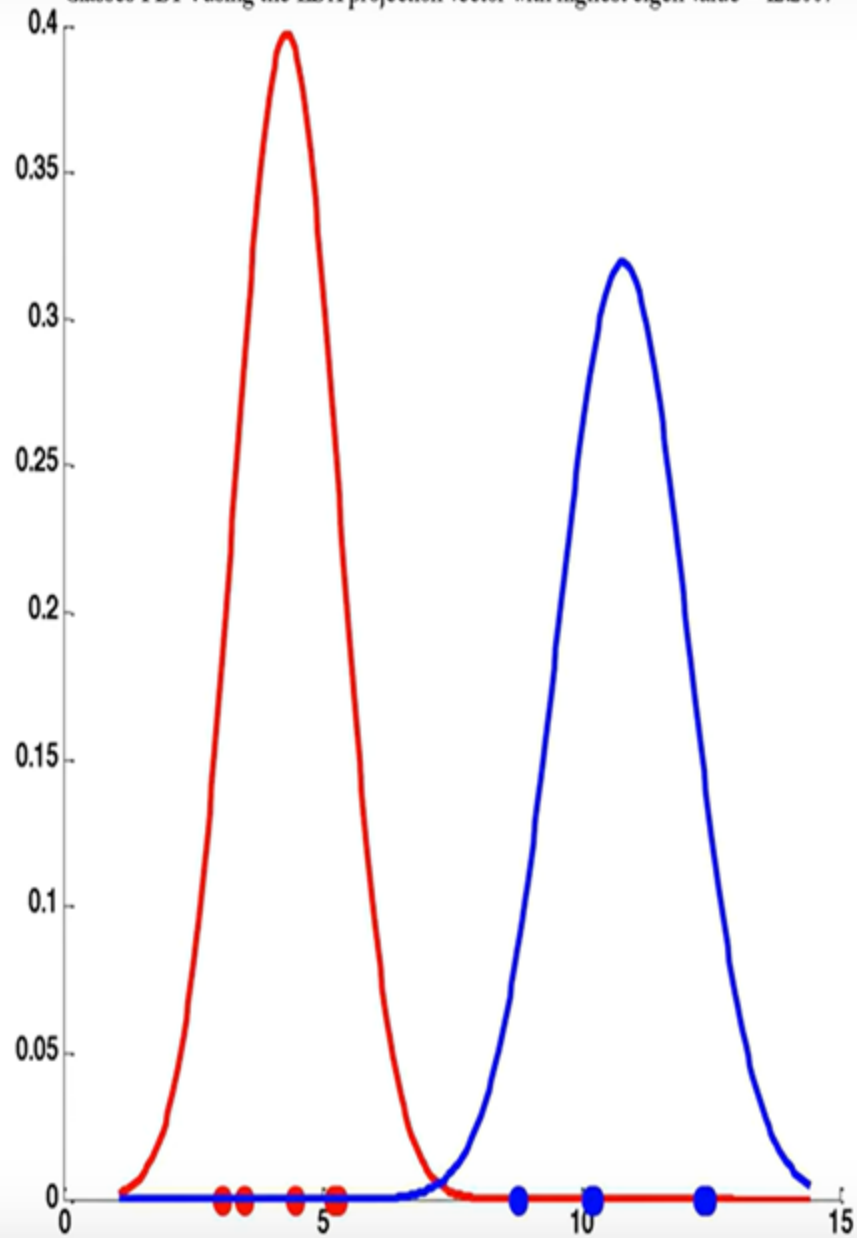
Sample for class ω_2 : $\mathbf{X}_2 = (x_1, x_2) = \{(9,10), (6,8), (9,5), (8,7), (10,8)\}$

X1	4	2	2	3	4	9	6	9	8	10
X2	2	4	3	6	4	10	8	5	7	8
1 st LD	4.46	3.48	3.06	5.2	5.3	12.35	8.8	10.2	10.19	12.42

LDA projection vector with the highest eigen value = 12.2007



Classes PDF : using the LDA projection vector with highest eigen value = 12.2007

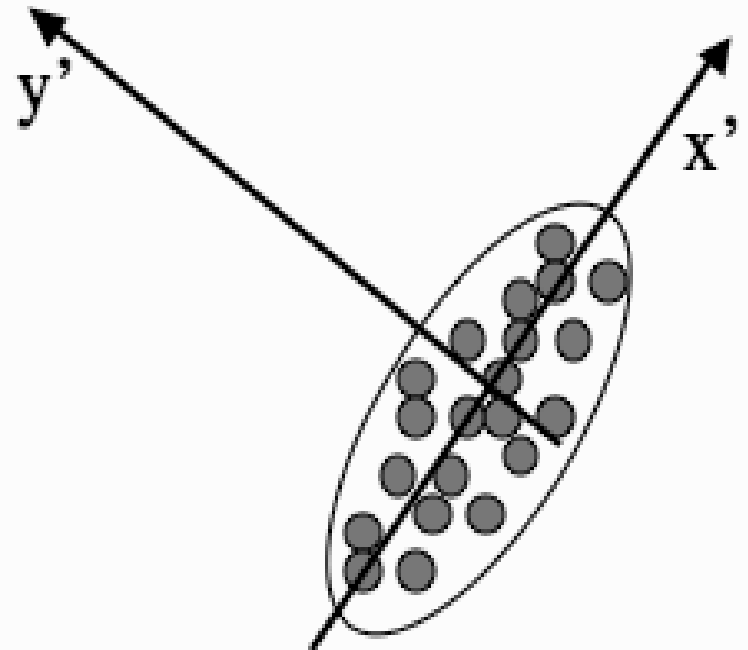
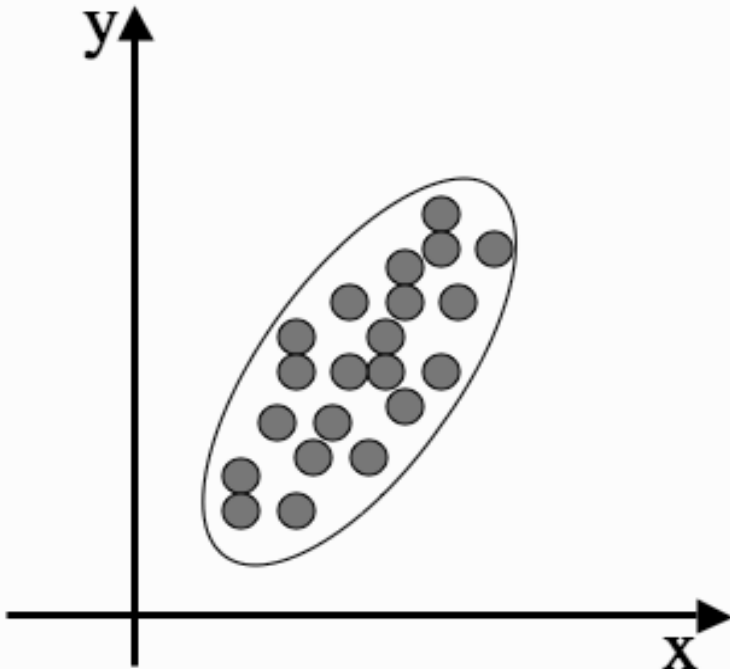


PRINCIPAL COMPONENTS ANALYSIS (PCA)

- Principal Component Analysis is an unsupervised learning algorithm that is used for the dimensionality reduction in machine learning
- Computing transformations of the data in order to identify a lower-dimensional set of axes.
- Unlike LDA, they are designed for unlabelled data.
- This does not stop them being used for labelled data, since the learning that takes place in the lower dimensional space can still use the target data, although it does mean that they miss out on any information that is contained in the targets.
- The idea is that by finding particular sets of coordinate axes, it will become clear that some of the dimensions are not required.
- This is demonstrated in Figure 6.6, which shows two versions of the same dataset.

- The potential for dimensionality reduction is in the fact that the *y dimension does not now demonstrate* much variability, and so it might be possible to ignore it and use the *x axis values alone* without compromising the results of a learning algorithm.
- In fact, it can make the results better, since we are often removing some of the noise in the data. (Translation, Rotation & Scaling)

FIGURE 6.6 Two different sets of coordinate axes. The second consists of a rotation and translation of the first and was found using Principal Components Analysis.



Based on the dataset find a new set of orthogonal feature vector in such a way that the data spread is maximum in the direction of feature vector (or dimension)

Figure 1

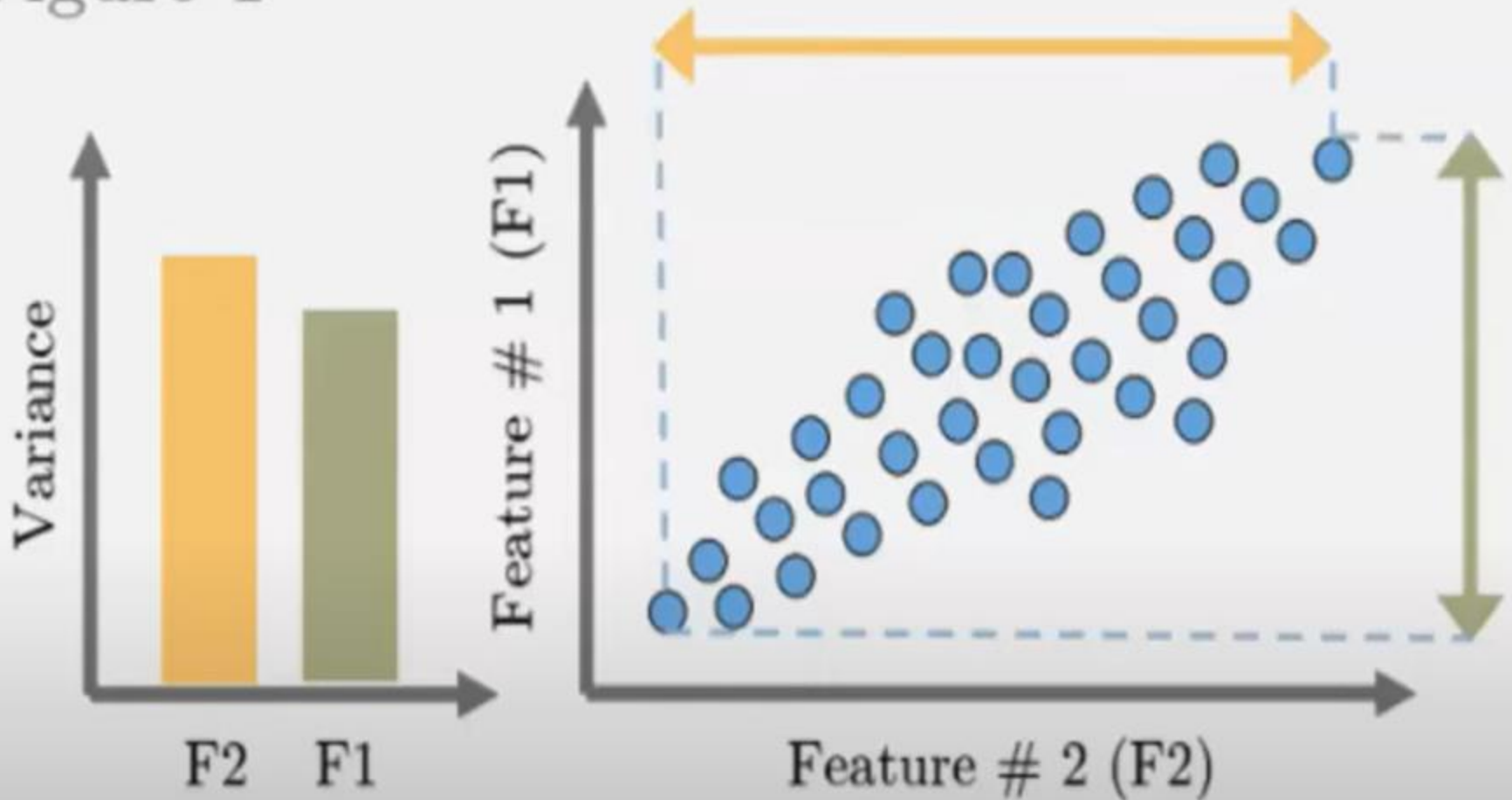


Figure 2

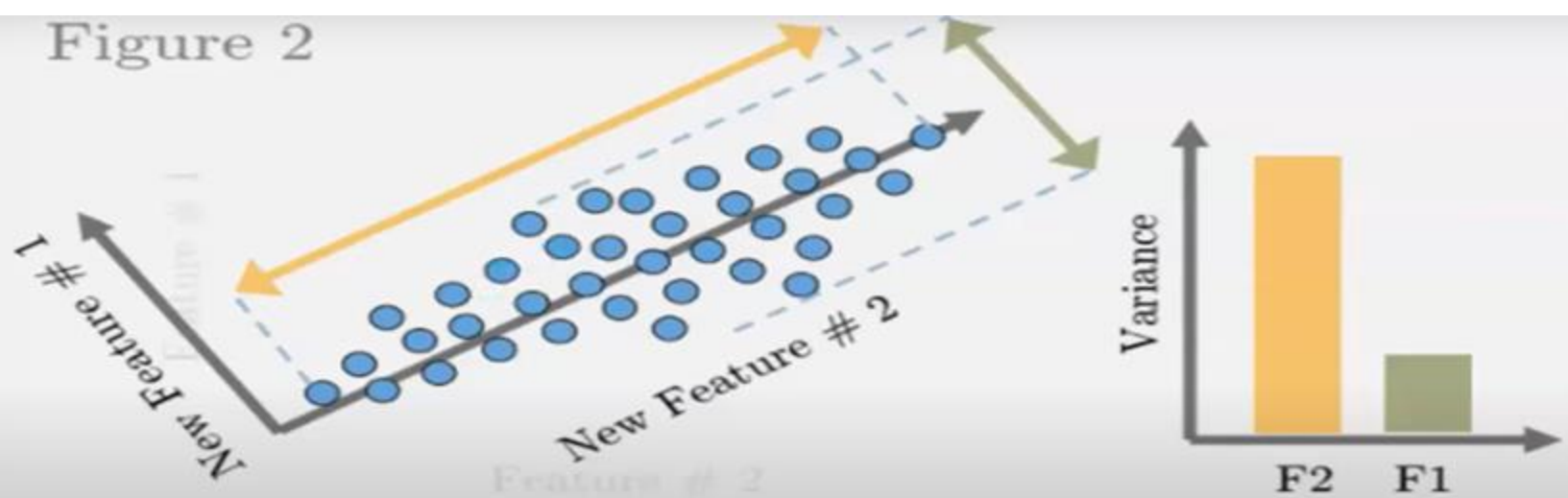
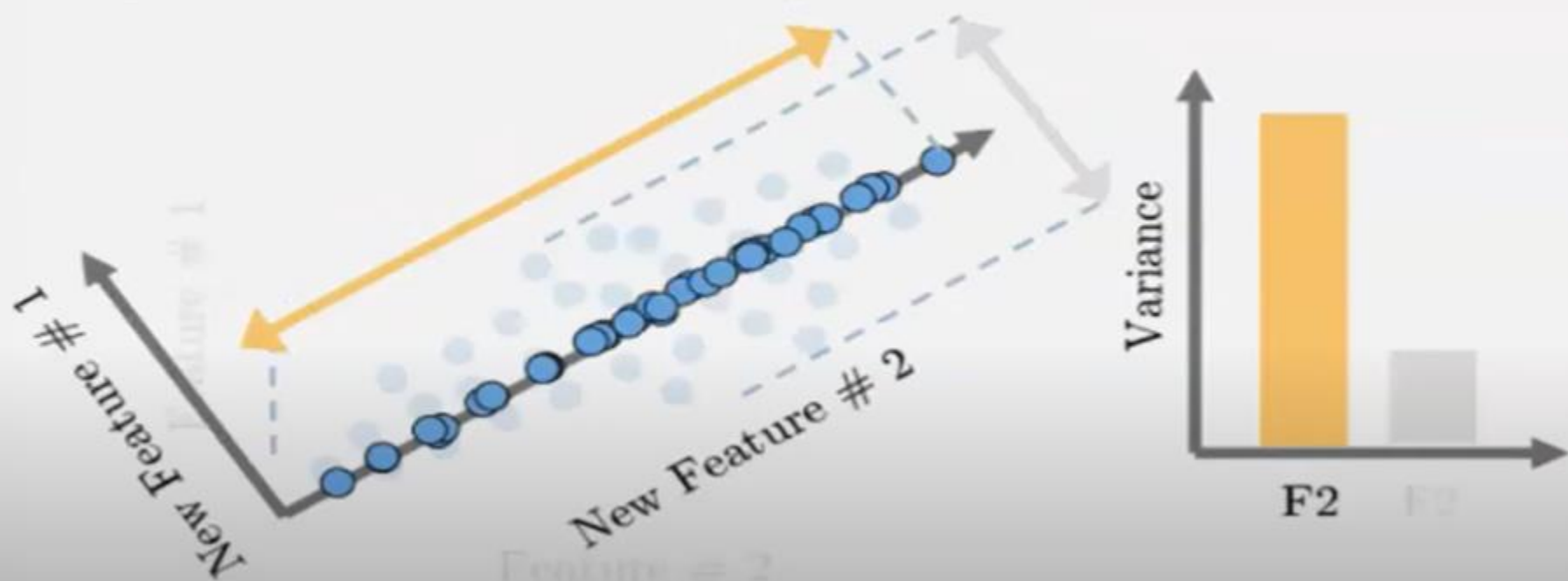


Figure 3



- The idea of a PCA is that it is a direction in the data with the largest variation.
- The algorithm first centres the data by subtracting off the mean, and then chooses the direction with the largest variation and places an axis in that direction, and then looks at the variation that remains and finds another axis that is orthogonal to the first and covers as much of the remaining variation as possible.
- It then iterates this until it has run out of possible axes.
- The end result is that all the variation is along the axes of the coordinate set, and so the covariance matrix is diagonal—each new variable is uncorrelated with every variable except itself.

Step 1. Data

- We consider a dataset having n features or variables denoted by $X_1; X_2; \dots ; \underline{X_n}$.
- Let there be N examples.
- Let the values of the i^{th} feature X_i be $X_{i1}; X_{i2}; \dots ; X_{iN}$

Features	Example 1	Example 2	...	Example N
X_1	X_{11}	X_{12}	...	X_{1N}
X_2	X_{21}	X_{22}	...	X_{2N}
$\vdots$				
X_i	X_{i1}	X_{i2}	...	X_{iN}
$\vdots$				
X_n	X_{n1}	X_{n2}	...	X_{nN}

Step 2. Compute the means of the variables

Features	Example 1	Example 2	...	Example N
X_1	X_{11}	X_{12}	...	X_{1N}
X_2	X_{21}	X_{22}	...	X_{2N}
$\vdots$				
X_i	X_{i1}	X_{i2}	...	X_{iN}
$\vdots$				
X_n	X_{n1}	X_{n2}	...	X_{nN}

$$\bar{X}_i = \frac{1}{N} (X_{i1} + X_{i2} + \dots + X_{iN})$$

Step 3. Calculate the covariance matrix

Features	Example 1	Example 2	...	Example N
X_1	X_{11}	X_{12}	...	X_{1N}
X_2	X_{21}	X_{22}	...	X_{2N}
$\vdots$				
X_i	X_{i1}	X_{i2}	...	X_{iN}
$\vdots$				
X_n	X_{n1}	X_{n2}	...	X_{nN}

$$\text{Cov}(X_i, X_j) = \frac{1}{N-1} \sum_{k=1}^N (X_{ik} - \bar{X}_i)(X_{jk} - \bar{X}_j)$$

$$S = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_n) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) & \cdots & \text{Cov}(X_2, X_n) \\ \vdots & & & \\ \text{Cov}(X_n, X_1) & \text{Cov}(X_n, X_2) & \cdots & \text{Cov}(X_n, X_n) \end{bmatrix}$$

Step 4. Calculate the eigenvalues and eigenvectors of the covariance matrix

- i. **Set up the equation:** This is a polynomial equation of degree n in λ . It has n real roots and these roots are the eigenvalues of S

$$\det(S - \lambda I) = 0.$$

- ii. **If $\lambda = \lambda'$ is an eigenvalue**, then the corresponding **eigenvector** is a vector

$$U = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix} \quad \text{such that} \quad (S - \lambda' I)U = 0$$

- iii. **We now normalize the eigenvectors.** Given any vector X we normalize it by dividing X by its length. The length (or, the norm) of the vector

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

is defined as

$$\|X\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

We compute the n normalised eigenvectors $e_1, e_2, \dots, e_n$ by

$$e_i = \frac{1}{\|U_i\|} U_i, \quad i = 1, 2, \dots, n.$$

Step 5. Derive new data set

- Order the eigenvalues from highest to lowest.
 - The unit eigenvector corresponding to the largest eigenvalue is the first principal component.
- i) Let the eigenvalues in descending order be $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ and let the corresponding unit eigenvectors be $e_1, e_2, \dots, e_n$.
 - ii) Choose a positive integer p such that $1 \leq p \leq n$.
 - iii) Choose the eigenvectors corresponding to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_p$ and form the following $p \times n$ matrix (we write the eigenvectors as row vectors):

$$F = \begin{bmatrix} e_1^T \\ e_2^T \\ \vdots \\ e_p^T \end{bmatrix}$$

Step 5. Derive new data set

iv) We form the following $n \times N$ matrix:

$$X = \begin{bmatrix} X_{11} - \bar{X}_1 & X_{12} - \bar{X}_1 & \cdots & X_{1N} - \bar{X}_1 \\ X_{21} - \bar{X}_2 & X_{22} - \bar{X}_2 & \cdots & X_{2N} - \bar{X}_2 \\ \vdots & & & \\ X_{n1} - \bar{X}_n & X_{n2} - \bar{X}_n & \cdots & X_{nN} - \bar{X}_n \end{bmatrix}$$

v) Next compute the matrix:

$$X_{\text{new}} = FX.$$

- Putting this in more formal terms, we have a data matrix $\mathbf{X}$ and want to rotate it so that the data lies along the directions of maximum variation.
- This means that we multiply our data matrix by a rotation matrix (often written as $\mathbf{P}^T$) so that $\mathbf{Y} = \mathbf{P}^T \mathbf{X}$, where $\mathbf{P}$ is chosen so that the covariance matrix of $\mathbf{Y}$ is diagonal, i.e.,

$$\text{cov}(\mathbf{Y}) = \text{cov}(\mathbf{P}^T \mathbf{X}) = \begin{pmatrix} \lambda_1 & 0 & 0 & \dots & 0 \\ 0 & \lambda_2 & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & \lambda_N \end{pmatrix}. \quad (6.7)$$

$$\text{cov}(\mathbf{Y}) = E[\mathbf{Y}\mathbf{Y}^T] \quad (6.8)$$

$$= E[(\mathbf{P}^T \mathbf{X})(\mathbf{P}^T \mathbf{X})^T] \quad (6.9)$$

$$= E[(\mathbf{P}^T \mathbf{X})(\mathbf{X}^T \mathbf{P})] \quad (6.10)$$

$$= \mathbf{P}^T E(\mathbf{X}\mathbf{X}^T) \mathbf{P} \quad (6.11)$$

$$= \mathbf{P}^T \text{cov}(\mathbf{X}) \mathbf{P}. \quad (6.12)$$

- The two extra things that we needed to know were that

$$(\mathbf{P}^T \mathbf{X})^T = \mathbf{P}^{TT} = \mathbf{X}^T \mathbf{P}$$

and that $E[\mathbf{P}] = \mathbf{P}$ (and obviously the same for $\mathbf{P}^T$) since it is not a data-dependent matrix. This then tells us that:

$$\begin{aligned} \mathbf{P} \mathbf{cov}(\mathbf{Y}) &= \mathbf{P} (\mathbf{P}^T \mathbf{cov}(\mathbf{X}) \mathbf{P}) \quad (\text{from equ. 6.12}) \\ &= \mathbf{cov}(\mathbf{X}) \mathbf{P} \quad \text{----} \quad (6.13) \end{aligned}$$

- where there is one tricky fact, namely that for a rotation matrix

$$\mathbf{P}^T = \mathbf{P}^{-1}$$

- Invert a rotation we rotate in the opposite direction by the same amount that we rotated forwards.
- As $\mathbf{cov}(\mathbf{Y})$ is diagonal, if we write $\mathbf{P}$ as a set of column vectors

- Find a matrix **P** so that for the directions that **P** is written in, the matrix **Z** does not twist or rotate those directions, but just rescales them.
- These directions are special enough that they have a name: they are **eigenvectors**, and the amount that they rescale the axes (the λ s) by are known as **eigen values**.
- All **eigenvectors** of a square symmetric matrix **A** are **orthogonal** to each other.
- The **eigenvectors** define a space. If we make a matrix **E** that contains the (normalized) **eigenvectors** of a matrix **A** as columns, then this matrix will take any vector and rotate it into what is known as the eigen space.

- Since $\mathbf{E}$ is a rotation matrix, $\mathbf{E}^{-1} = \mathbf{E}^T$, so that rotating the resultant vector back out of the **eigen space** requires multiplying it by $\mathbf{E}^T$, where by 'normalized', means **eigenvectors** are made unit length.
- What should we do between rotating the vector into the **eigen space**, and rotating it back out? The answer is that we can stretch the vectors along the axes.
- This is done by multiplying the vector by a diagonal matrix that has the **eigen values** along its diagonal, $\mathbf{D}$.
- So we can decompose any square symmetric matrix $\mathbf{A}$ into the following set of matrices: $\mathbf{A} = \mathbf{E}\mathbf{D}\mathbf{E}^T$, and this is what we have done to our covariance matrix above.
- This is called the spectral decomposition.

The Principal Components Analysis Algorithm

- Write N datapoints $\mathbf{x}_i = (x_{1i}, x_{2i}, \dots, x_{Mi})$ as row vectors
- Put these vectors into a matrix $\mathbf{X}$ (which will have size $N \times M$)
- Centre the data by subtracting off the mean of each column, putting it into matrix $\mathbf{B}$
- Compute the covariance matrix $\mathbf{C} = \frac{1}{N} \mathbf{B}^T \mathbf{B}$
- Compute the eigenvalues and eigenvectors of $\mathbf{C}$, so $\mathbf{V}^{-1} \mathbf{C} \mathbf{V} = \mathbf{D}$, where $\mathbf{V}$ holds the eigenvectors of $\mathbf{C}$ and $\mathbf{D}$ is the $M \times M$ diagonal eigenvalue matrix
- Sort the columns of $\mathbf{D}$ into order of decreasing eigenvalues, and apply the same order to the columns of $\mathbf{V}$
- Reject those with eigenvalue less than some η , leaving L dimensions in the data

-
- Two different examples of using PCA are shown in Figures 6.7 and 6.8.
 - The former shows two-dimensional data from an ellipse being mapped into one principal component, which lies along the principal axis of the ellipse.
 - Figure 6.8 shows the first two dimensions of the iris data, and shows that the three classes are clearly distinguishable after PCA has been applied.

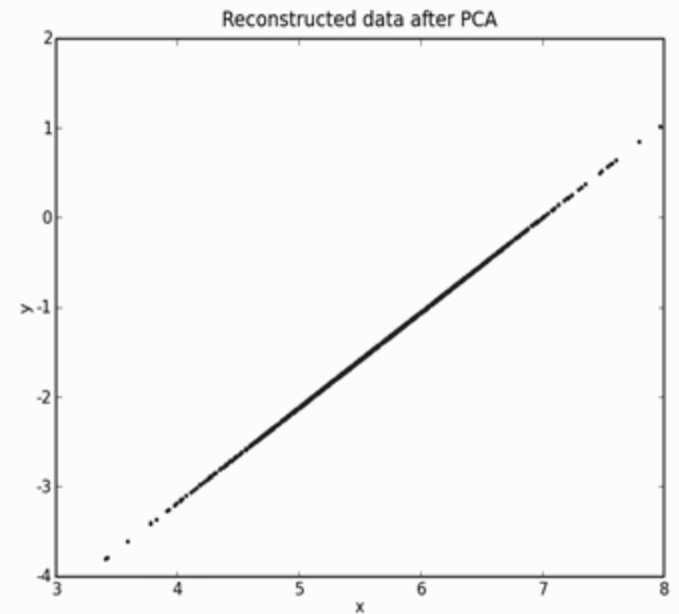
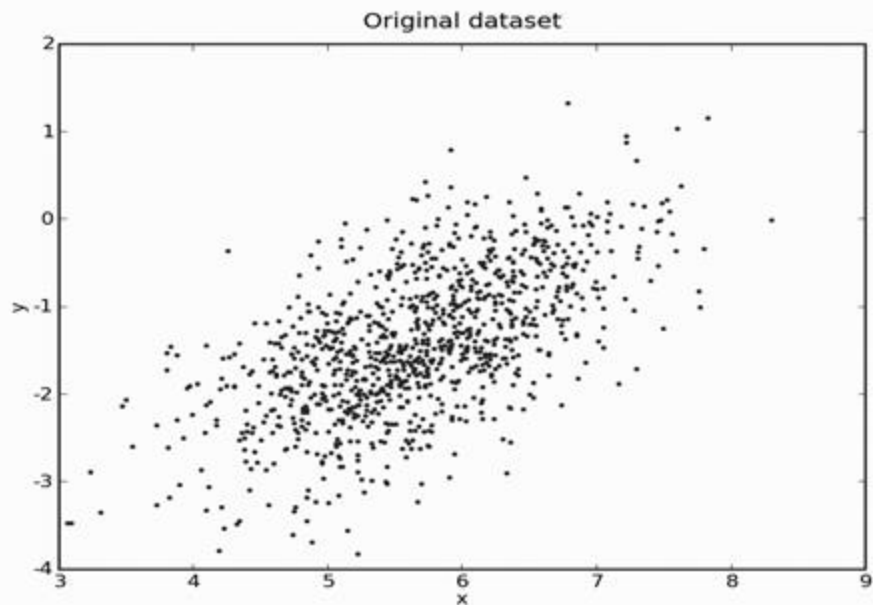


FIGURE 6.7 Computing the principal components of the 2D dataset on the left and using only the first one to reconstruct it produces the line of data shown on the right, which is along the principal axis of the ellipse that the data was sampled from.



FIGURE 6.8 Plot of the first two principal components of the iris data, showing that the three classes are clearly distinguishable

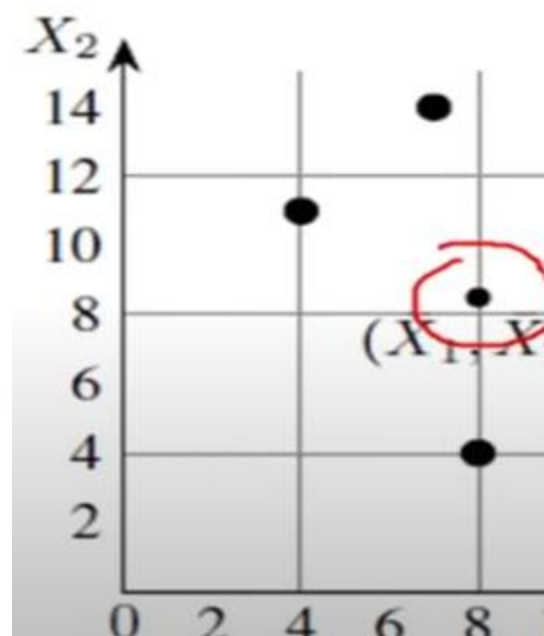
- Given the data in Table, reduce the dimension from 2 to 1 using the Principal Component Analysis (PCA) algorithm.

Feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

Step 1: Calculate Mean

$$\bar{X}_1 = \frac{1}{4}(4 + 8 + 13 + 7) = 8,$$

$$\bar{X}_2 = \frac{1}{4}(11 + 4 + 5 + 14) = 8.5.$$



Step 2: Calculation of the covariance matrix.

$$S = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) \end{bmatrix}$$

$$\begin{aligned} \text{Cov}(X_1, X_1) &= \frac{1}{N-1} \sum_{k=1}^N (X_{1k} - \bar{X}_1)(X_{1k} - \bar{X}_1) \\ &= \frac{1}{3} ((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2) \\ &= 14 \end{aligned}$$

$$\begin{aligned} \text{Cov}(X_1, X_2) &= \frac{1}{N-1} \sum_{k=1}^N (X_{1k} - \bar{X}_1)(X_{2k} - \bar{X}_2) \\ &= \frac{1}{3} ((4-8)(11-8.5) + (8-8)(4-8.5) \\ &\quad + (13-8)(5-8.5) + (7-8)(14-8.5)) \\ &= -11 \end{aligned}$$

$$\text{Cov}(X_2, X_1) = \text{Cov}(X_1, X_2) \\ = -11$$

$$\begin{aligned} \text{Cov}(X_2, X_2) &= \frac{1}{N-1} \sum_{k=1}^N (X_{2k} - \bar{X}_2)(X_{2k} - \bar{X}_2) \\ &= \frac{1}{3} \left((11 - 8.5)^2 + (4 - 8.5)^2 + (5 - 8.5)^2 + (14 - 8.5)^2 \right) \\ &= 23 \end{aligned}$$

$$\text{covariance matrix} = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Step 3: Eigenvalues of the covariance matrix

The characteristic equation of the covariance matrix is,

$$\begin{aligned}0 &= \det(S - \lambda I) \\&= \begin{vmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{vmatrix} \\&= (14 - \lambda)(23 - \lambda) - (-11) \times (-11) \\&= \lambda^2 - 37\lambda + 201\end{aligned}$$

$$\begin{aligned}\lambda &= \frac{1}{2}(37 \pm \sqrt{565}) \\&= 30.3849, 6.6151 \\&= \lambda_1, \lambda_2 \quad (\text{say})\end{aligned}$$

Step 4: Computation of the eigenvectors

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = (S - \lambda I) U$$

$$U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$= \begin{bmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$= \begin{bmatrix} (14 - \lambda)u_1 - 11u_2 \\ -11u_1 + (23 - \lambda)u_2 \end{bmatrix}$$

$$\begin{aligned} (14 - \lambda)u_1 - 11u_2 &= 0 \\ -11u_1 + (23 - \lambda)u_2 &= 0 \end{aligned}$$

$$\frac{u_1}{11} = \frac{u_2}{14 - \lambda} = t$$

$$u_1 = 11t, \quad u_2 = (14 - \lambda)t$$

$$U_1 = \begin{bmatrix} 11 \\ 14 - \lambda \end{bmatrix}.$$

- To find a unit eigenvector, we compute the length of U_1 which is given by,

$$\begin{aligned} \|U_1\| &= \sqrt{11^2 + (14 - \lambda_1)^2} \\ &= \sqrt{11^2 + (14 - 30.3849)^2} \\ &= 19.7348 \end{aligned}$$

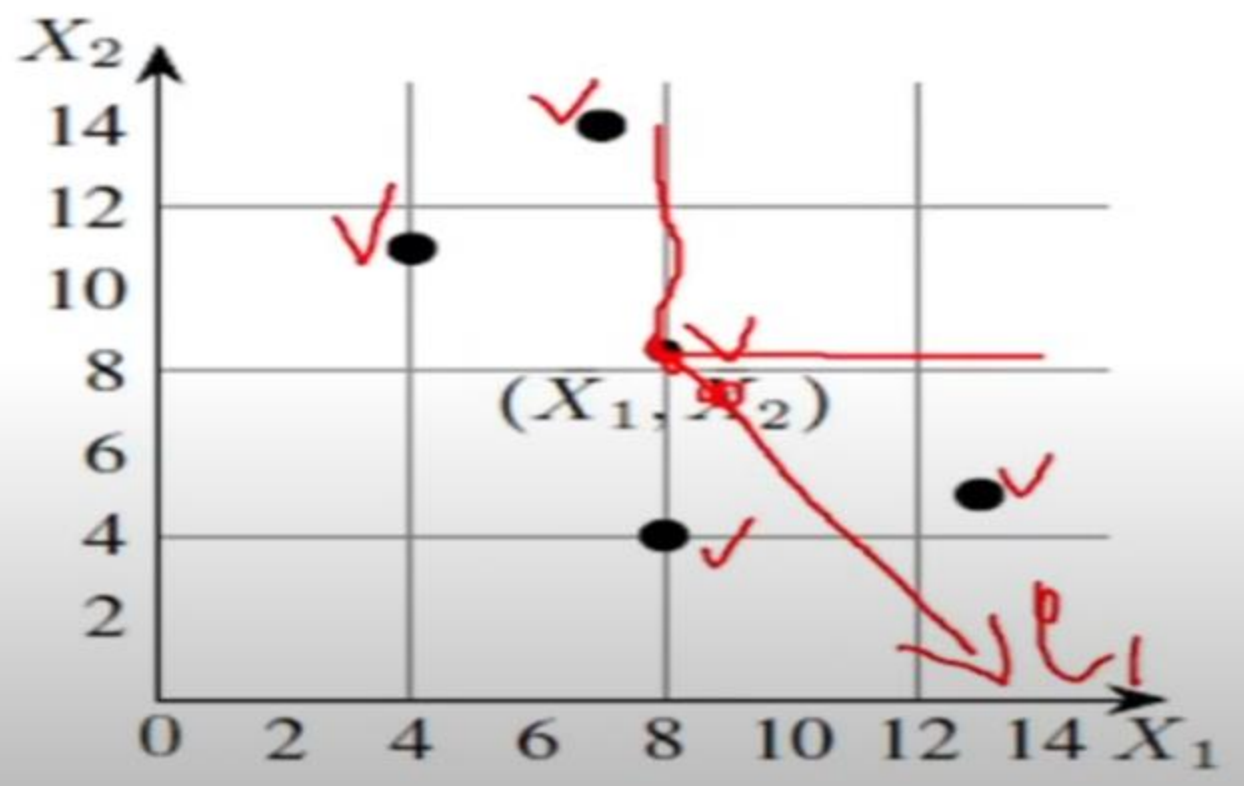
$$\begin{aligned}
 e_1 &= \begin{bmatrix} 11/\|U_1\| \\ (14 - \lambda_1)/\|U_1\| \end{bmatrix} \\
 &= \begin{bmatrix} 11/19.7348 \\ (14 - 30.3849)/19.7348 \end{bmatrix} \\
 &= \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}
 \end{aligned}$$

Step 5: Computation of first principal components

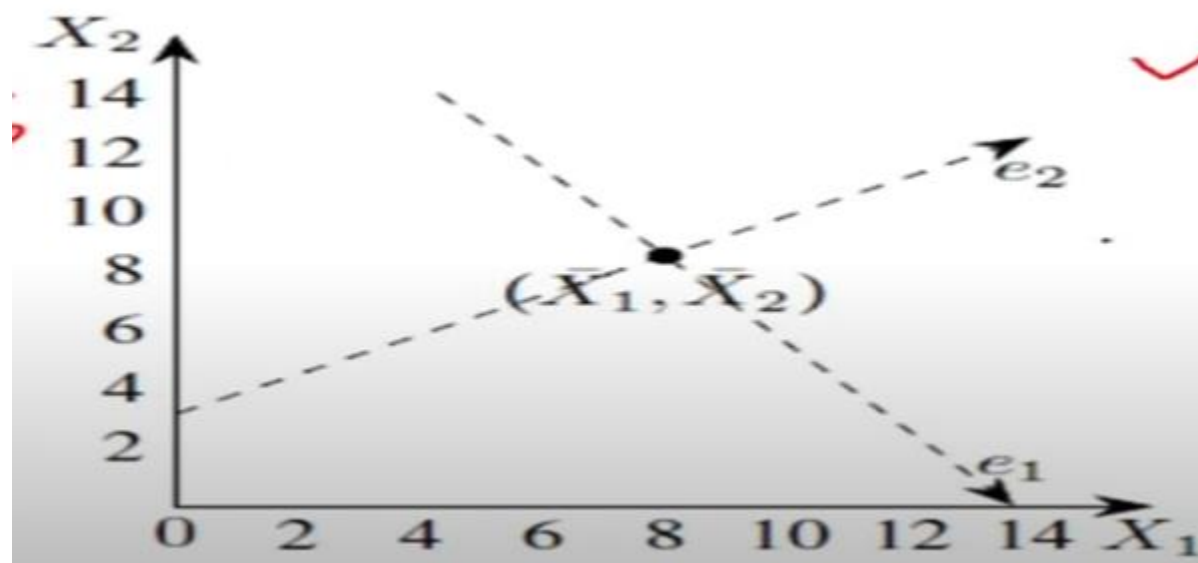
$$e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix}$$

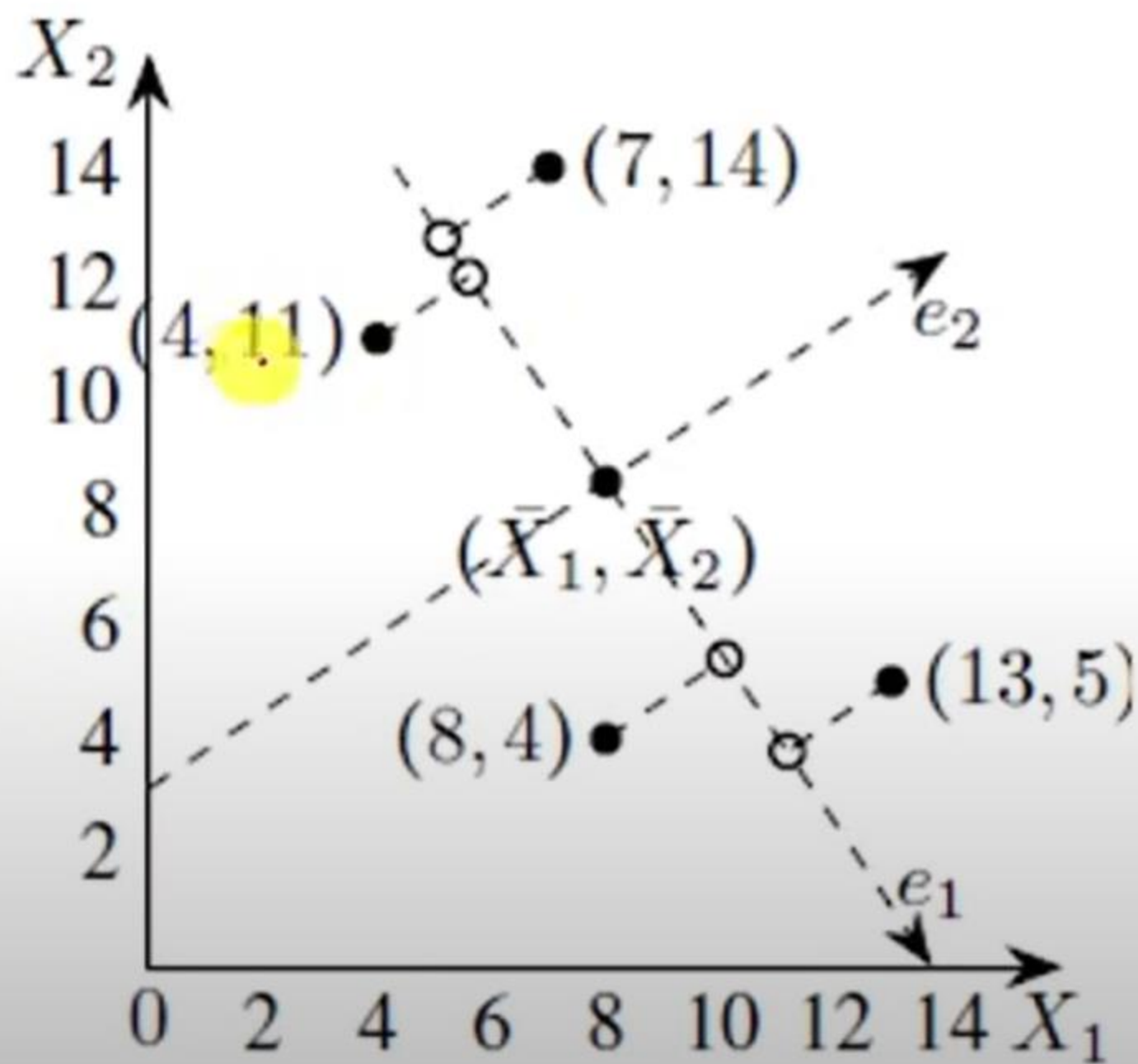
$$\begin{aligned}
 e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix} &= \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} X_{11} - \bar{X}_1 \\ X_{21} - \bar{X}_2 \end{bmatrix} \\
 &= 0.5574(X_{11} - \bar{X}_1) - 0.8303(X_{21} - \bar{X}_2) \\
 &= 0.5574(4 - 8) - 0.8303(11 - 8, 5) \\
 &= -4.30535
 \end{aligned}$$

Feature	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14
First Principle Components	-4.3052	3.7361	5.6928	-5.1238



$$e_1 = \begin{bmatrix} 0.557 \\ -0.830 \end{bmatrix}$$





FACTOR ANALYSIS

- It is a Statistical Method used to describe variability among observed / correlated variables
- The task of factor analysis is to group the variables that are most related or correlated
- The idea of it is whether the data can be explained by a smaller number of uncorrelated factors or latent variables.
- The problem of factor analysis is to find those independent factors, and the noise that is inherent in the measurements of each factor.
- A dataset is in the usual $N \times M$ matrix $\mathbf{X}$, i.e., each row of $\mathbf{X}$ is an $\mathbf{M}$ dimensional data point, and $\mathbf{X}$ has **covariance matrix**

- Write the model as: $\mathbf{X} = \mathbf{WY} + \boldsymbol{\epsilon}$ --- (6.20)
- where $\mathbf{X}$ are the observations and $\boldsymbol{\epsilon}$ is the noise.
- Since the factors $\mathbf{b}_i$ that we want to find should be independent, so $\text{cov}(\mathbf{b}_i, \mathbf{b}_j) = \mathbf{0}$ if $i \neq j$.
- Factor analysis takes explicit notice of the noise in the data, using the variable $\boldsymbol{\epsilon}$.
- It assumes that the noise is Gaussian with zero mean and some known variance: $\boldsymbol{\Psi}$ (psi), with the variance of each element being

$$\Psi_i = \text{var}(\boldsymbol{\epsilon}_i)$$
- Noise measurements are independent of each other, as the data come from a set of separate (independent) physical processes.

- Aim of factor analysis is to try to find a set of factor loadings $\mathbf{W}_{ij}$ and values for the variance of the noise parameters Ψ .
- Data in $\mathbf{X}$ can be reconstructed from the parameters, or so that we can perform dimensionality reduction.
- Adding additional variables, the natural formulation is an **EM** algorithm and this is how the computations are usually performed to produce the maximum likelihood estimate.
- Getting to the EM algorithm takes some effort.
- Replace several of the terms with values, and ignore any terms that do not depend on θ .

- EM algorithm we have to differentiate this with respect to $\mathbf{W}$ and the individual elements of Ψ , and apply some linear algebra, to get update rules.

$$\mathbf{W}_{new} = (\mathbf{y}E(\mathbf{x}|\mathbf{y})^T) (E(\mathbf{x}\mathbf{x}^T|\mathbf{y}))^{-1}, \quad (6.23)$$

$$\Psi_{new} = \frac{1}{N} \text{diagonal} (\mathbf{x}\mathbf{x}^T - \mathbf{W}E(\mathbf{x}|\mathbf{y})\mathbf{y}^T), \quad (6.24)$$

- where $\text{diagonal}()$ ensures that the matrix retains values only on the diagonal and the expectations are:

$$E(\mathbf{x}|\mathbf{y}) = \mathbf{W}^T(\mathbf{W}\mathbf{W}^T + \Psi)^{-1}\mathbf{b} \quad (6.25)$$

$$E(\mathbf{x}\mathbf{x}^T|\mathbf{x}) - E(\mathbf{x}|\mathbf{y})E(\mathbf{x}|\mathbf{y})^T = \mathbf{I} - \mathbf{W}^T(\mathbf{W}\mathbf{W}^T + \Psi)^{-1}\mathbf{W}. \quad (6.26)$$

- Need to add to the algorithm is some way to decide when to stop, which involves computing the log likelihood and stopping the algorithm when it stops descending.

INDEPENDENT COMPONENTS ANALYSIS (ICA)

- There is a related approach to factor analysis that is known as Independent Components Analysis.
- In PCA, the components were chosen so that they were orthogonal and uncorrelated (so that the covariance matrix was diagonal, i.e., so $\text{cov}(\mathbf{b}_i, \mathbf{b}_j) = 0$ if $i \neq j$).
- The components are statistically independent ($E[\mathbf{b}_i, \mathbf{b}_j] = E[\mathbf{b}_i]E[\mathbf{b}_j]$ as well as the $\mathbf{b}_i$ being uncorrelated), then we get **ICA**.
- The common motivation for **ICA** is the problem of blind source separation.
- As with factor analysis, the assumption is that the data we see are actually created by a set of underlying physical processes that are independent.

- The most popular way to describe blind source separation is known as the cocktail party problem.
- If you are at a party, then your ears hear lots of different sounds coming from lots of different locations (different people talking, the clink of glasses, background music, etc.) but you are somehow able to focus on the voice of the people you are talking to, and can in fact separate out the sounds from all of the different sources even though they are mixed together.
- The cocktail party problem is the challenge of separating out these sources, although there is one wrinkle: for the algorithm to work, you need as many ears as there are sources.

- This is because the algorithm does not have the information we have about what things sound like.
- Suppose that we have two sources making noise (S_1^t , S_2^t) where the top index covers the fact that there are lots of data points appearing over time, and two microphones that hear things, giving inputs (x_1^t , x_2^t). The sounds that are heard come from the sources as:

$$x_1 = as_1 + bs_2 \quad \text{--- (6.27)}$$

$$x_2 = cs_1 + ds_2 \quad \text{--- (6.28)}$$

- which can be written in matrix form as: $\mathbf{x} = \mathbf{A}\mathbf{s}$ ---- (6.29)
- where $\mathbf{A}$ is known as the **mixing matrix**. $\mathbf{s} = \mathbf{A}^{-1}\mathbf{x}$.
- Except that, unfortunately, we don't know $\mathbf{A}$. The approximation to $\mathbf{A}^{-1}$ that is generally labelled as $\mathbf{W}$, it is a square matrix as same number of microphones as we do sources.
- Need to work out what we actually know about the sources and the signals.

LOCALLY LINEAR EMBEDDING

- **Locally Linear Embedding (LLE)** is a method of **Non Linear** Dimensionality reduction
- Is an unsupervised approach designed to transform data from its original high-dimensional space into a lower-dimensional representation,
- The first tries to approximate the data by sticking together sets of locally flat patches that cover the dataset, while the second uses the shortest distances (geodesics) on the non-linear space to find a globally optimal solution.
- First look at locally linear algorithm, which is called Locally Linear Embedding (LLE).
- By making linear approximations some errors will be made, so make these errors as small as possible by making the patches small where there is lots of non-linearity in the data.

- The error is known as the reconstruction error and is simply the sum-of-squares of the distance between the original point and its reconstruction:

$$\epsilon = \sum_{i=1}^N \left(\mathbf{x}_i - \sum_{j=1}^N \mathbf{W}_{ij} \mathbf{x}_j \right)^2. \quad (6.31)$$

- The weights $\mathbf{W}_{ij}$ say how much effect the j^{th} data point has on the reconstruction of the i^{th} one.
- The question is which points can be usefully used to reconstruct a particular data point.
- If another point is a long way off, then it probably isn't very useful: only those points that are close to the current data point are used.

- The **k nearest points** are **neighbours**
- Solving for the weights $\mathbf{W}_{ij}$ is a **least-squares problem**, which we can simplify by enforcing the constraints that for any point $\mathbf{x}_j$ that is a long way from the current point $\mathbf{x}_i$, $\mathbf{W}_{ij} = \mathbf{0}$
- This produces a reconstruction of the data, but it does not reduce the dimensionality at all.

$$\sum_j \mathbf{w}_{ij} = 1$$

- For this, reapply the same basic cost function, but minimize it according to the positions $\mathbf{y}_i$ of the points in some lower dimensional space (dimension L):

$$y_i = \sum_{i=1}^N \left(y_i - \sum_{j=1}^L \mathbf{W}_{ij} y_j \right)^2. \quad (6.32)$$

ISOMAP

- It is a variant of the standard multi-dimensional scaling (MDS) algorithm.

MULTI-DIMENSIONAL SCALING (MDS)

- Like PCA, MDS tries to find a linear approximation to the full data space that embeds the data into a lower dimensionality.
- In the case of MDS the embedding tries to preserve the distances between all pairs of points.
- Start with data points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N \in \mathbf{R}^M$.
- Choose a new dimensionality $L < M$ and compute the embedding so that the data points are $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_N \in \mathbf{R}^L$.

- Need a cost function to minimize.
- Lots of choices for MDS cost functions, but the more common ones are:
- **Kruskal–Shephard scaling (also known as least-squares)**
- **Sammon mapping**
- This puts more weight onto short distances, so that neighbouring points stay the correct distance apart.
- In either case, gradient descent can be used to minimize the distances.
- It is possible to construct a direct algorithm that does not have to use gradient descent.
- This classical MDS algorithm works fine on flat manifolds (data spaces).
- Interested in manifolds that are not flat, and this is where Isomap comes in.

- The algorithm has to construct the distance matrix for all pairs of data points on the manifold, but there is no information about the manifold, and so the distances can't be computed exactly.
- It builds up the distances between points that are far away by finding paths that run through points that are close together, i.e., that are neighbours, and then uses normal MDS on this distance matrix: Floyd's and Dijkstra's algorithms are well-known algorithms for finding shortest paths on graphs.
- They are of $O(N^3)$ and $O(N^2)$ time complexity, respectively.
- There is one practical aspect of Isomap, which is that getting the number of neighbours right can be important, otherwise the graph splits into separate components (that is, segments of the graph that are not linked to each other), which have infinite distance between them.

- Be careful to deal only with the largest component, which means that you end up with less data than you started with.
- By increasing the neighbourhood size over 50, so that each point had more neighbours than were in its class, the results shown in the figure were produced.

Evolutionary Learning

- Need to view Evolution as a search problem.
- Animals are competing with each other in all kinds of ways—for example, eating each other—which encourages them to try to find camouflage colours, become toxic to certain predators, etc.
- Evolution works on a population through an imaginary fitness landscape, which has an implicit bias towards animals that are ‘fitter’, i.e., those animals that live long enough to reproduce, are more attractive, and so get more mates, and generate more and healthier offspring.
- The Genetic Algorithm models the Genetic process that gives rise to Evolution.
- Both parents give some genetic information to their off-spring.
- In Fig. 10.1, in biological organisms, each parent passes on one chromosome out of their two, and so there is a 50% chance of any gene making it into the offspring.
- Of the two versions of each gene (one from each parent) one allele (variation) is selected.

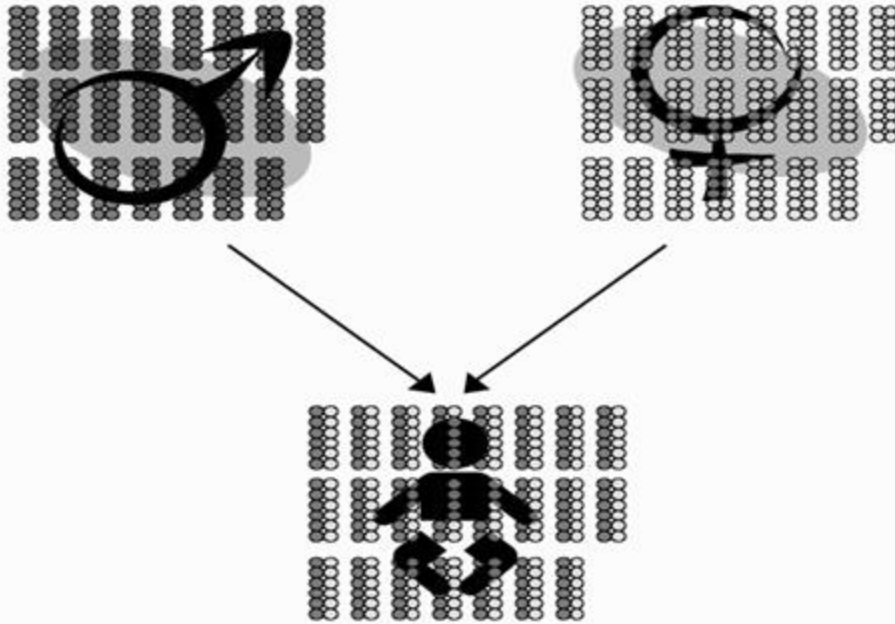


FIGURE 10.1 Each adult in the mating pair passes one of their two chromosomes to their offspring.

- Of the two versions of each gene (one from each parent) one allele (variation) is selected.
- Hence, children have similarities with their parents, and there is lots of genetic inheritance.
- However, there are also random mutations, caused by copying errors when the chromosome material is reproduced, which means that some things do change over time.

- The genetic algorithm shows many of the things that are best and worst about machine learning: it is often, but not always, very effective, it has an array of parameters that are crucial, but hard to set, and it is impossible to guarantee that it will find a result that is always good at all.
- Genetic algorithms perform both Exploitation and Exploration.
- Make incremental improvements to current good solutions, but also find radically new solutions, some of which may be better than the current best.
- A variation of the genetic algorithm that acts on trees that represent computer programs that is known as Genetic Programming, and a set of algorithms that use sampling from a probability distribution rather than an evolving population in order to find better solutions.

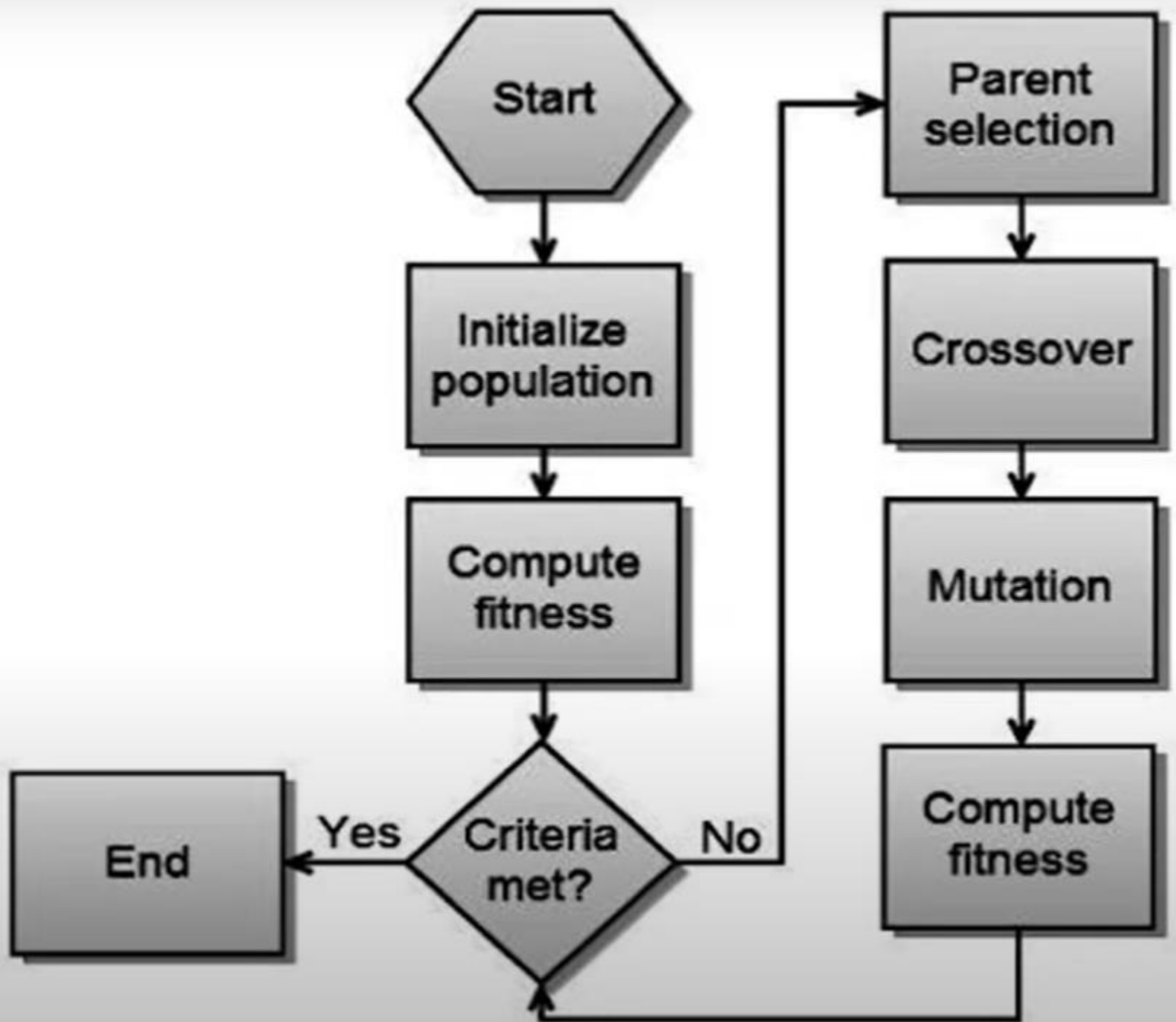
THE GENETIC ALGORITHM (GA)

- Genetic algorithm is a class of evolutionary algorithm
- GA is a search technique used in computing to find approximate solutions to optimization problems and search problems.
- This algorithm reflects the process of natural selection where the fittest individuals are selected for reproduction in order to produce offspring of the next generation.
- It is a Computational Approximation to how evolution performs search, which is by producing modifications of the parent genomes in their offspring and thus producing new individuals with different fitness.
- Neuron—it attempts to abstract away everything except the important parts that we need to understand what evolution does.

THE GENETIC ALGORITHM (GA)

- From this principle, model simple genetics inside a computer and solve problems with it are:
 - a method for representing problems as chromosomes
 - a way to calculate the fitness of a solution
 - a selection method to choose parents
 - a way to generate offspring by breeding the parents

Applications: Image processing, Designing electronic circuits, code-breaking, artificial creativity



- Ex: NP-complete problem (runs in exponential time or worse in the number of inputs) known as the knapsack problem and is easy to describe, but difficult to solve in general.
- To measure how much space it takes up and then write a program to work out how to fill as much of the bag as possible, so that you get the best value for your airfare.
- This problem, and variations of it, appear in various disguises in Cryptography, Combinatorics, Applied Mathematics, Logistics, and Business, so it is an important problem.
- Since it is NP-complete, finding the optimal solution for interesting cases (pretty much anything above 10 items) is computationally impossible.

Suppose that you are packing for your holidays.

- You've bought the biggest and best rucksack that was for sale, but there is still no way that you are going to fit in everything you want to take (camera, money, addresses of friends, etc.) and the things that your mum is insisting you take (spare dress, phrasebook, stamps to write home with, etc.).
- Greedy algorithm that finds solutions to the knapsack problem.
- At each stage it takes the largest thing that hasn't been packed yet and that will still fit into the bag, and iterates that rule.
- Not necessarily return the optimal solution(unless each thing is larger than the sum of all the ones smaller than it, in which case it will), but it is very quick and simple.
- GA should be getting a much better solution than the greedy rule most of the time to be worth all the effort involved in writing and running it.

STRING REPRESENTATION

- The individual solutions are represented in analogy to the chromosome.
- GAs use a string, with each element of the string (gene) being chosen from some alphabet (Binary).
- The different values in the alphabet, are analogous to the alleles(series of genes on a chromosome determine the hereditary characteristics).
- The alphabet for the knapsack problem is very simple, make it binary.
- Make the string L units long, where L is *the total number of things and make* each unit a binary digit.

- One option would be to set the fitness to 0 if the things in that string will not all fit.
- Make the fitness function be the sum of the values of the items to be taken if they fit into the knapsack, but if they do not we will subtract twice the amount by which they are too big for the knapsack from the size of the knapsack.
- Allows solutions that are only just over to be considered for improvement, but tries to ensure that they are not the fittest solution

POPULATION

- The GA works on a population of strings, with the first generation usually being created randomly around.
- The fitness of each string is then evaluated, and that first generation is bred together to make a second generation, which is then used to generate a third, and so on.

- Encode a solution: 0 for the things not taken & 1 for things taken
- Four things to take: $(0, 1, 1, 0)$ means that take the middle two, but not the first or last.
- It does not tell us whether or not this string is possible nor whether it is a good string (whether it fills the knapsack).
- Need some way to decide how well each string fulfills the problem criteria and is known as the fitness of the string.

EVALUATING FITNESS

- The fitness function can be seen as an oracle that takes a string as an argument and returns a value for that string.
- Together with the string encoding the fitness function forms the problem-specific part of the GA.

- After the initial population is chosen randomly, the algorithm evolves to produce each successive generation, with the hope being that there will be progressively fitter individuals in the populations as the number of generations increases.
- To make the initial population for the knapsack problem, we will now create a set of random binary strings of length L *by using the random number generator*.
- Choose parents out of this population, and start breeding them.

GENERATING OFFSPRING: PARENT SELECTION

- The idea here is that average fitness will improve if we select strings that are already relatively fit compared to the other members of the population (following natural selection), which is exploitation of our current population.

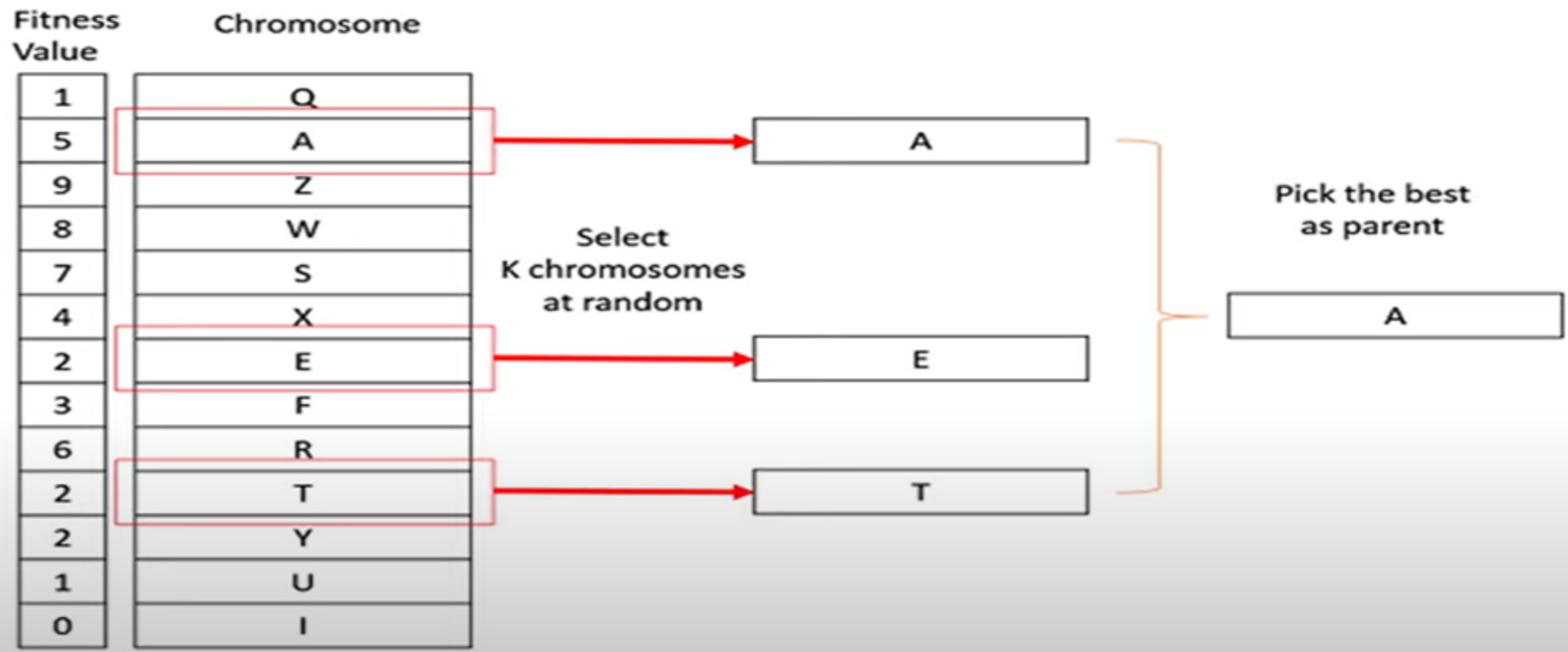
- Allow some possibility of weak strings being considered.
- Fitter strings are more likely to be chosen to enter the ‘mating pool’.

Three employed ways to produce better results:

- **Tournament Selection:** Repeatedly pick four strings from the population, with replacement and put the fittest two of them into the mating pool.
 - We select k individuals from the population at random and select the best out of these to become a parent
- **Truncation Selection:** Pick some fraction f of the best strings and ignore the rest. For example, $f = 0.5$ is often used, so the best 50% of the strings are put into the mating pool, each twice so that the pool is the right size.
- The pool is randomly shuffled to make the pairs.

- It is a method used for large population/mass selection
- In truncation selection, individuals are sorted according to their fitness
- Only the best individuals are selected as parents
- Truncation threshold $trunc$ is used as the parameter for truncation selection

Tournament Selection



- **Fitness Proportional Selection:** The probability of a string being selected being proportional to its fitness.
- The function that is generally used is (for string):

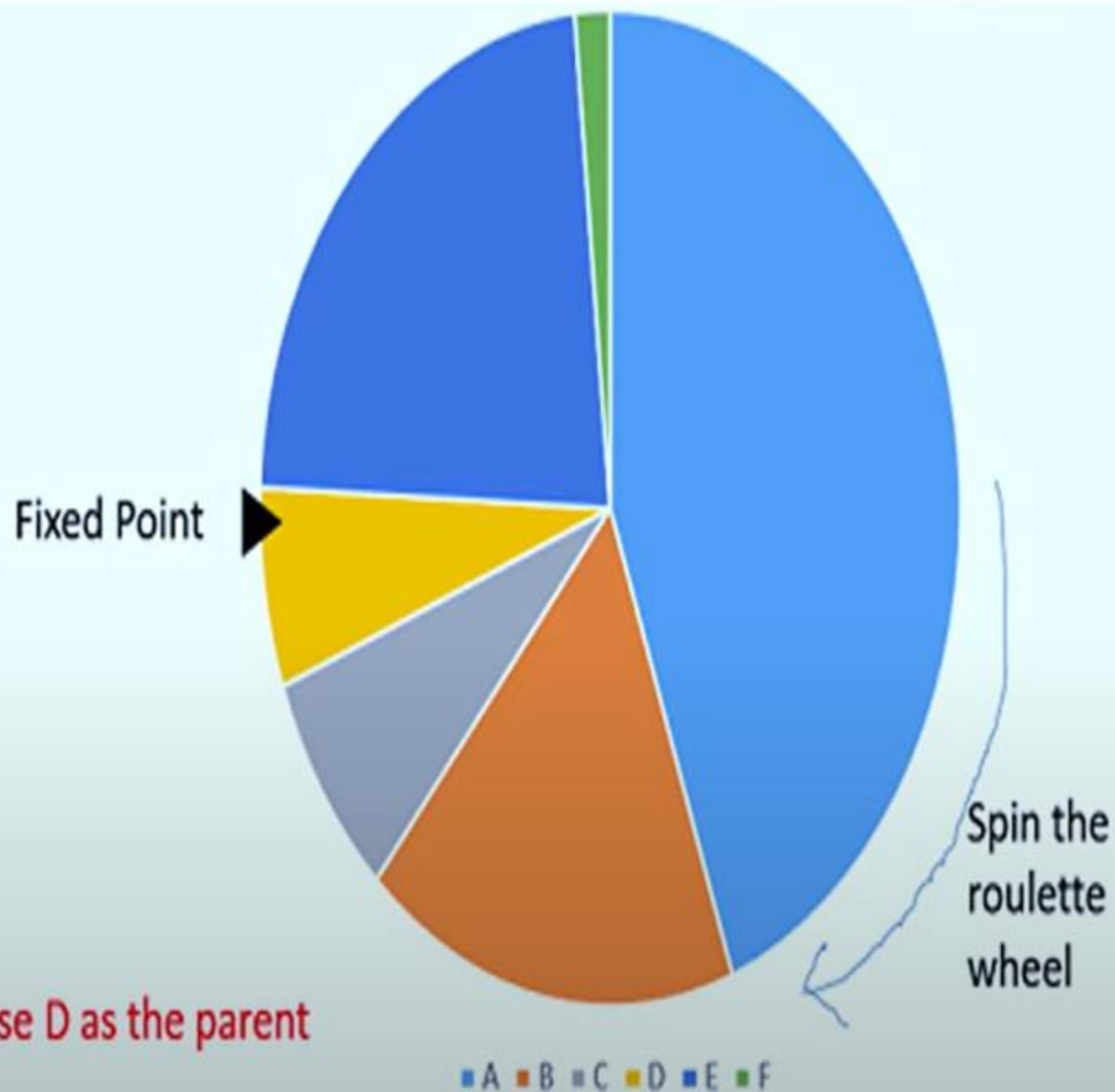
$$p^{\alpha} = \frac{F^{\alpha}}{\sum_{\alpha'} F^{\alpha'}}, \quad (10.1)$$

- where F^{α} is the fitness. If the fitness is not positive then F needs to be replaced by $\exp(sF)$ throughout, where s is the selection strength, a parameter, and the equation is the soft-max activation.

$$p^{\alpha} = \frac{\exp(sF^{\alpha})}{\sum_{\alpha'} \exp(sF^{\alpha'})}. \quad (10.2)$$

- One way around this is to add more copies of the fitter strings, so that they are more likely to get chosen.
- This is sometimes called ‘Roulette Selection’

Chromosome	Fitness Value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3



- If each string gets an area on a roulette wheel, then the larger the area associated to one number, the more likely it is that the ball will land there.
- Randomly pick strings from this larger set.
- Populate much larger new Population array, which contains multiple copies of each string.
- Select the strings to put into the mating pool, the next operation is to put them into pairs.
- Since the order that they are in is random, we can simply pair up the strings so that each even-indexed string takes the following odd-indexed one as its mate.

GENERATING OFFSPRING: GENETIC OPERATORS

- **Having selected our breeding pairs, now need to decide how to combine their two strings to generate the offspring, which is the genetics part of the algorithm.**
- There are two genetic operators that are generally used.

CROSSOVER

- In biology, organisms have two chromosomes, and each parent donates one of them.
- Members of our GA population only have one chromosome-equivalent, the string.
- Crossover is the operator that performs global exploration, since the strings that are produced are radically different to both parents in at least some places.

- Generate the new string as part of the first parent and part of the second.
- Pick one point at random in the string, and to use parent 1 for the first part of the string, up to the crossover point and parent 2 for the rest.
- Generate two offspring, with the second one consisting of the first part of parent 2 and the second part of parent 1.
- known as single point crossover, and the extension to multi-point crossover is hopefully obvious.
- ‘Extreme’ version is known as uniform crossover and consists of independently selecting each element of the string at random from the two parents.
- The three types of crossover are shown in Figure 10.2.

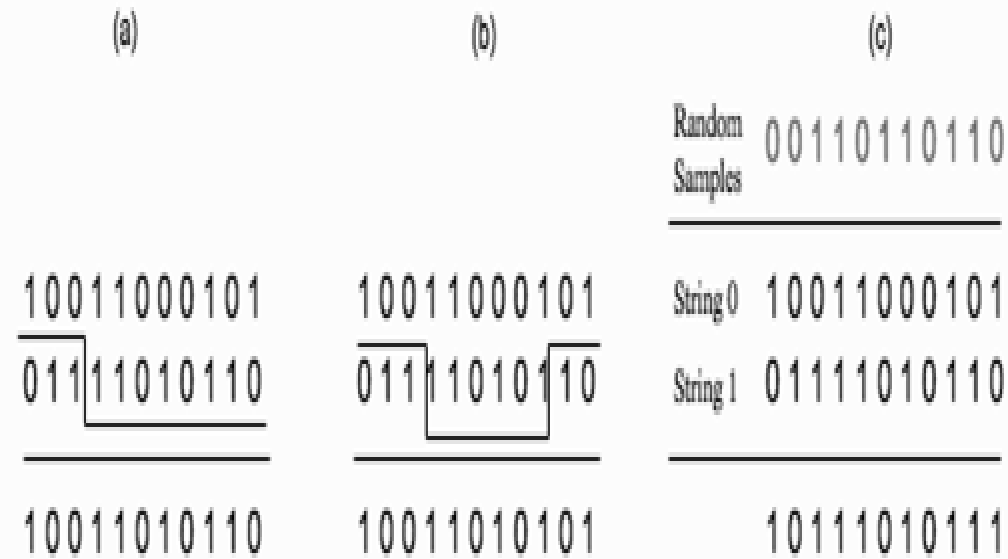


FIGURE 10.2 The different forms of the crossover operator. (a) Single point crossover. A position in the string is chosen at random, and the offspring is made up of the first part of parent 1 and the second part of parent 2. (b) Multi-point crossover. Multiple points are chosen, with the offspring being made in the same way. (c) Uniform crossover. Random numbers are used to select (0-String0 & 1-String1) which parent to take each element from.

Crossover is not always useful, depending upon the problem; for example, in the Travelling Salesman Problem, the strings that are generated by crossover might not even be valid tours.

GAs work well on problems where the solution comes from putting together lots of little solutions, so that different strings assemble each separate building block, and then crossover puts those substrings together to make the final solution.

- **Example:** Imagine a bird that has webbed feet for good swimming, but that cannot fly, breeding with a bird that can fly, but not swim.
- The offspring? A duck! Obviously, this is not biologically plausible, but it is a good picture of how crossover works.
- One interesting feature of the GA that obviously isn't true in real genetics is that in addition to the duck the algorithm would produce the bird that can't fly or swim.
- Exceptions to this, such as the great New Zealand Kiwi, which can neither swim nor fly, but is happily not extinct.

MUTATION

- The other genetic operator is mutation, which effectively performs local random search.
- The value of any element of the string can be changed, governed by some (usually low) probability p .
- *For our binary alphabet in the knapsack problem, mutation causes a bit-flip, as is shown in Figure 10.3.*
- For chromosomes with real values, some random number is generally added or subtracted from the current value.
- Often, $p \sim 1/L$ where L is the string length, so that there is approximately one mutation in each string.

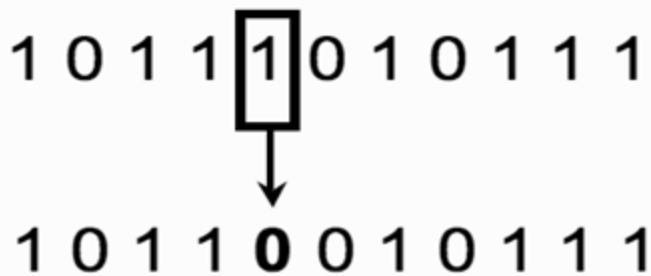


FIGURE 10.3 The effects of mutation on a string.

ELITISM, TOURNAMENTS, AND NICHING

- Produce pairs of offspring by taking pairs of parents
- Replace the parents by their children to make a completely new population, and carry on from there.
- The maximum fitness in each generation can decrease, at least temporarily, and since in the end we are only interested in the ‘best’ solution, this seems a bit risky: we could potentially lose a really good string that we find early on in the search, and that we never see again.
- **ELITISM:** Takes some number of the fittest strings from one generation and puts them directly into the next population, replacing strings that are already there either at random, or by choosing the least fit to replace.

- At every iteration the population stays the same size, something else that is unlike real evolution.
- **TOURNAMENT:** Two parents and their two off-springs compete, with the two fittest out of the four being put into the new population.
- Implementation of these functions continues along the same lines as the previous ones;
- Elitism and Tournaments both ensure that good solutions aren't lost, they both have the problem that they can encourage premature convergence, where the algorithm settles down to a constant population that never changes even though it hasn't found an optimum.
- GA favours fitter members of the population, which means that a solution that reaches a local maximum will generally be favoured, and this solution will be exploited.

- Tournaments and Elitism encourage this, because they reduce the amount of diversity in the population by allowing the same individuals to remain over many generations.
- By this Exploration aspect of the GA stops occurring.
- Exploration will be downplayed, making it hard to escape from the local maximum— strings of worse fitness will be replaced in the population.
- The randomness in the GA is a very large part of why it works, and schemes to reduce that randomness often harm the overall results.
- **NICHING:** Used to solve the problem of premature convergence also known as using island populations, where the population is separated into several subpopulations, which all evolve independently for some period of time.
- So that they are likely to have converged to different local maxima, and a few members of one subpopulation are occasionally injected as ‘immigrants’ into another subpopulation.

The Basic Genetic Algorithm

- Initialisation
 - generate N random strings of length L with the chosen alphabet
- Learning
 - repeat:
 - * create an (initially empty) new population
 - * repeat:
 - select two strings from current population, preferably using fitness-proportional selection
 - recombine them in pairs to produce two new strings
 - mutate the offspring
 - either add the two offspring to the population, or use tournaments to put two strings from the four of parents and offspring into the population
 - * until N strings for the new population are generated
 - * optionally, use elitism to take the fittest strings from the parent generation and replace some others from the child generation
 - * keep track of the best string in the new population
 - * replace the old population with the new one
 - until stopping criteria met

USING GENETIC ALGORITHMS

MAP COLOURING

- Graph colouring is a typical Discrete Optimization Problem.
- Colour a graph using only k colours, and choose them in such a way that adjacent regions have different colours.
- Mathematically proven that any two-dimensional planar graph can be coloured with four colours.
- Even though it is impossible, we try to solve the three-colour problem using a genetic algorithm.
- To apply a genetic algorithm, there are three basic tasks that need to be performed:

Encode possible solutions as strings

- Choose our alphabet to consist of the three possible shades (black (b), dark (d), and light (l), say).
- For a six-region map, a possible string is $\alpha = \{bdbllbb\}$.

- This says that the first region is black, the second dark grey, etc.
- We choose an order to record the regions in and stick to it for all the strings, and now we can encode any way of colouring in those six regions.
- An example problem and a colouring are given in Figures 10.4 and 10.5.

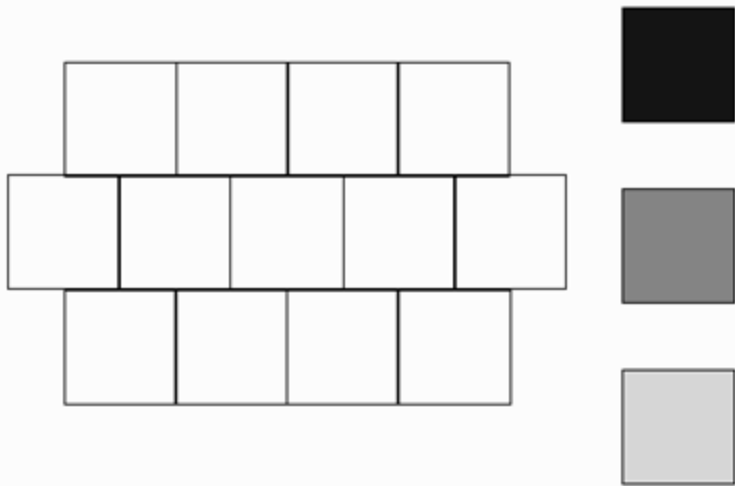


FIGURE 10.4 A sample map that we wish to colour using the three colours shown, without any two adjacent squares having the same colour.

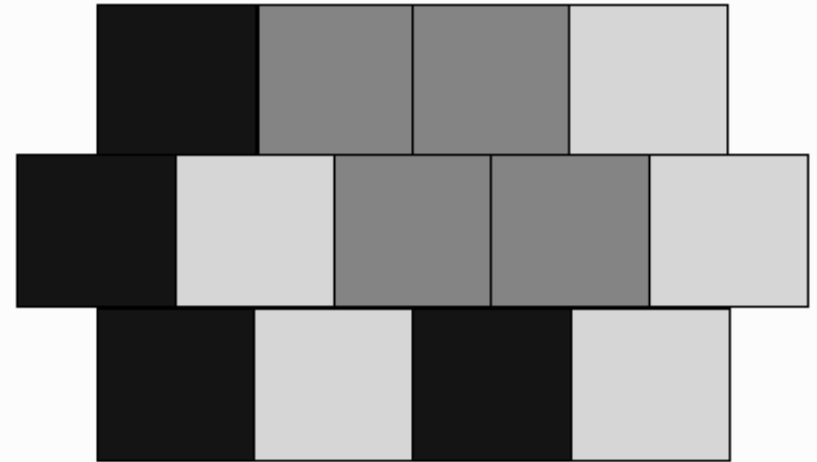


FIGURE 10.5 A possible colouring with several adjacent squares having the same colour.

CHOOSE A SUITABLE FITNESS FUNCTION

- Minimize the cost function is the number of times that two adjacent regions have the same colour.
- The best solution has the lowest number, not the highest.
- Multiply all the scores by minus one and use Equation (10.2) to turn them into fitnesses, or to count the total number of lines between regions and subtract off the number where the two regions on either side of the line have the same colour.
- Count the number of correct edges.
- The Ex. in Fig.10.5 has 16 out of the 26 boundaries correct (where a boundary is the intersection between any two squares), so its fitness is 16.

$$p^{\alpha} = \frac{\exp(sF^{\alpha})}{\sum_{\alpha'} \exp(sF^{\alpha'})}$$

CHOOSE SUITABLE GENETIC OPERATORS

- Use the standard genetic operators - crossover and mutation.
- The way that they are used is shown in Figures 10.6 and 10.7.

Having made those choices, we can let the GA run on the problem, with a possible population and their offspring shown in Figure 10.8, and look at the best solutions after some preset number of iterations. The GA produces good solutions to this problem.

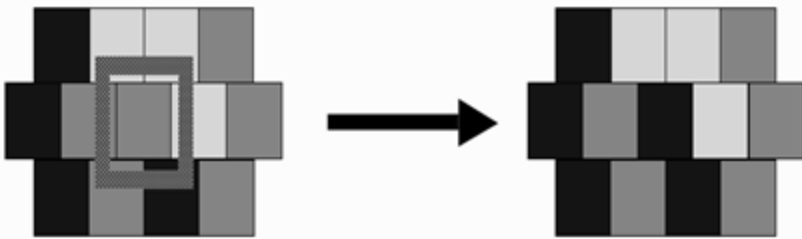


FIGURE 10.6 The way that mutation is performed on a colour, changing it into one of the other colours.

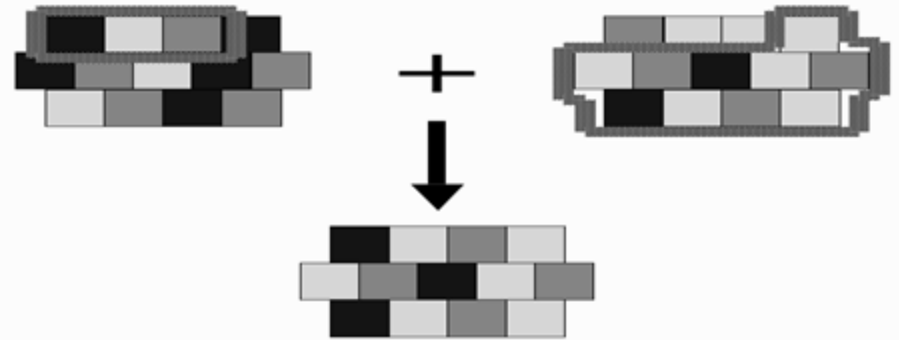


FIGURE 10.7 The effects of crossover on a

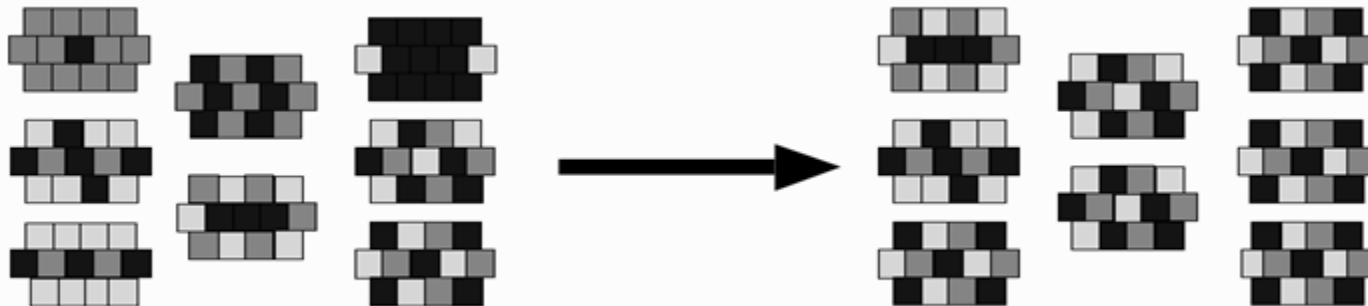


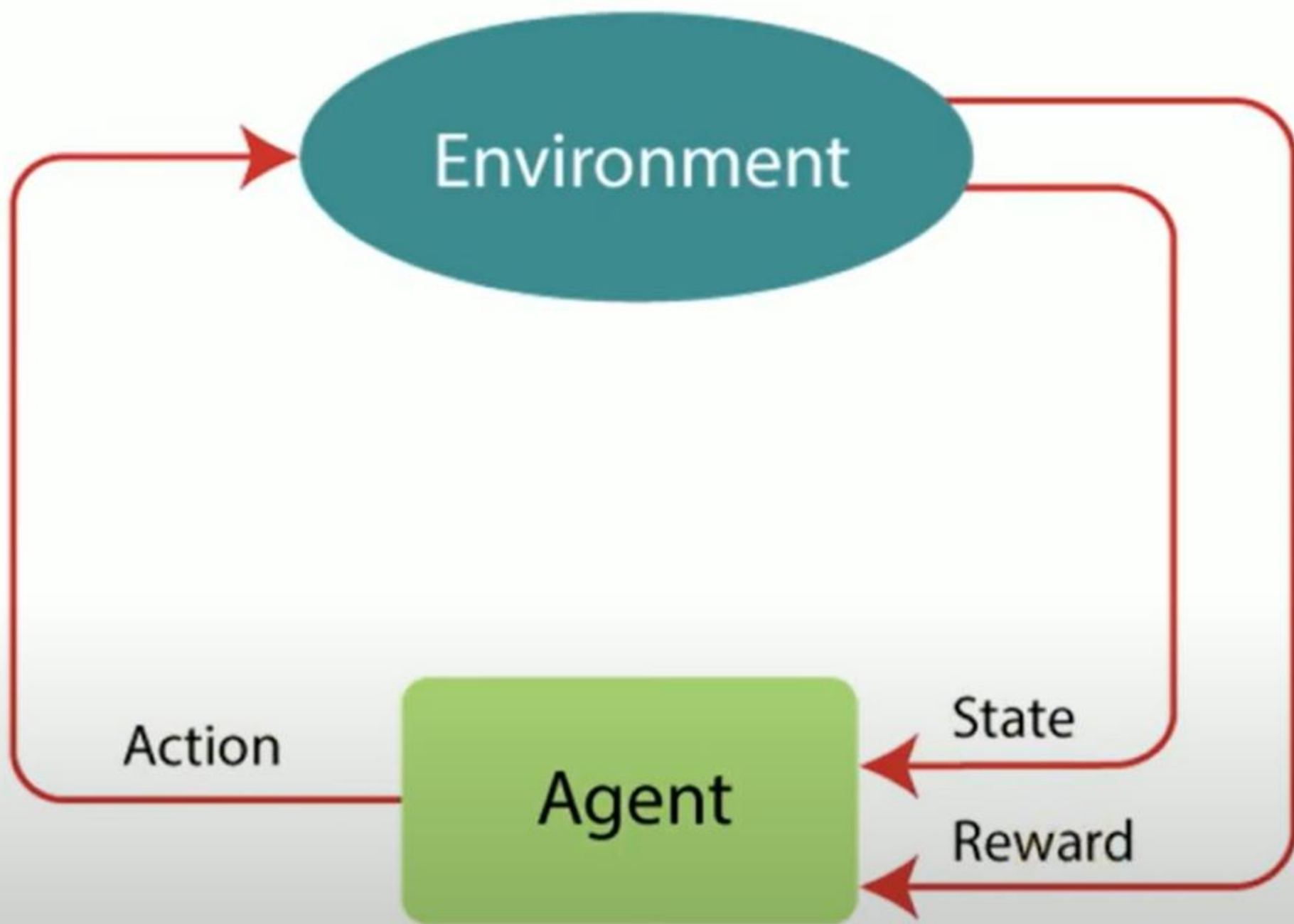
FIGURE 10.8 One generation of the GA working on the map colouring problem.

Limitations of the GA

- A significant one of which is they can be very slow.
- Once a local maximum has been reached, it can often be a long time before a string is produced that escapes from the local maximum and finds another, higher, maximum.
- Do not know anything about the fitness landscape
- It is very hard to analyse the behaviour of the GA
- Cannot guarantee that the algorithm will converge at all, and certainly not to the optimal solution.
- Treated as a black box

REINFORCEMENT LEARNING

- It fills the gap between supervised learning, where the algorithm is trained on the correct answers given in the target data, and unsupervised learning, where the algorithm can only exploit similarities in the data to cluster it.
- It tries out different strategies and see which work best.
- Is a feed-back based machine learning approach
- **Search is a fundamental part of any reinforcement learner:** the algorithm searches over the state space of possible inputs and outputs in order to try to maximize a reward.
- Described in terms of the interaction between some agent and its environment.
- The agent is the thing that is learning, and the environment is where it is learning, and what it is learning about.
- Environment provides information about how good a strategy is, through some reward function.



A

B

C

D

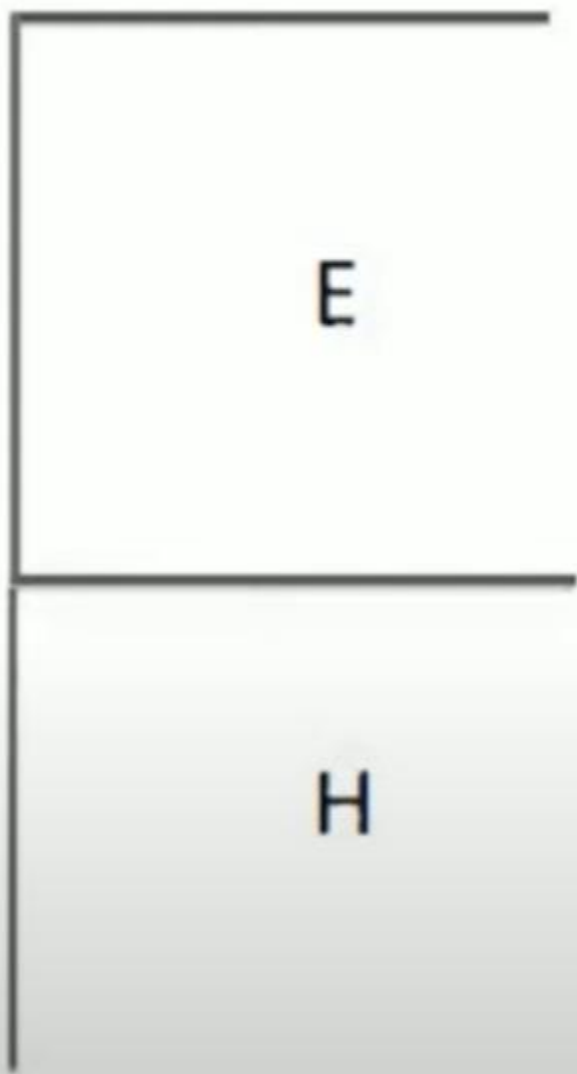
E

F

G

H

I



For each correct action, the agent receives positive feedback, and for each incorrect action, the agent gets negative feedback (penalty).

OVERVIEW

- Reinforcement learning maps states to actions in order to maximize some numerical reward.
- Algorithm knows about the current input (the state), and the possible things it can do (the actions), and its aim is to maximize the reward.
- The current sensor readings of the robot could define the state.
- State doesn't necessarily tell us everything that it would be useful to know (sensors don't tell it its location, only what it can see about it), and there can be noise and inaccuracies in the state data.
- Possible ways of robot can drive its motors are the actions, it moves the robot in the environment, and the reward could be how well it does its task without crashing into things.

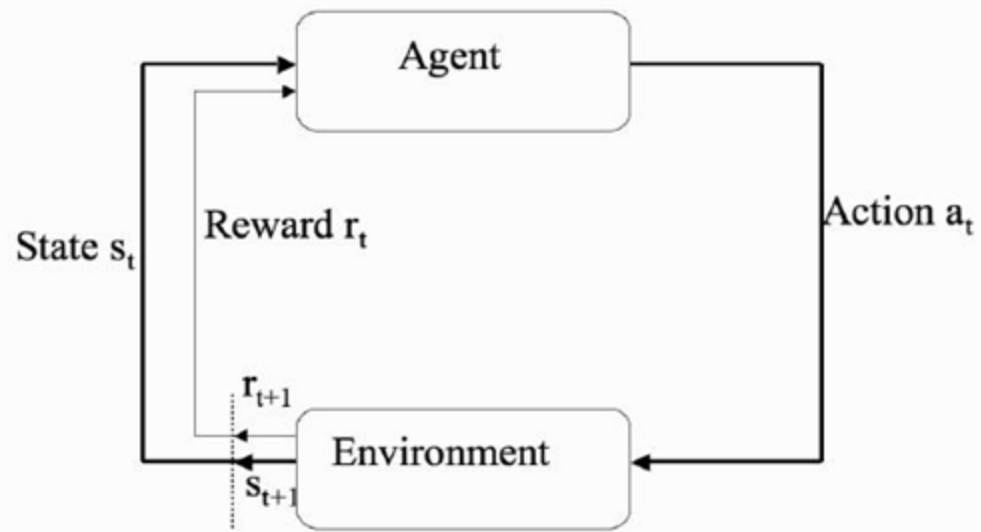


FIGURE 11.1 A robot perceives the current state of its environment through its sensors, and performs actions by moving its motors. The reinforcement learner (agent) within the robot tries to predict the next state and reward.

FIGURE 11.2 The reinforcement learning cycle: the learning agent performs action a_t in state s_t and receives reward r_{t+1} from the environment, ending up in state s_{t+1}

- Algorithm gets feedback in the form of the reward about how well it is doing.
- In contrast to supervised learning, where the algorithm is ‘taught’ the correct answer, the reward function evaluates the current solution, but does not suggest how to improve it.

- To make the situation a little more difficult, need to think about the possibility that the reward can be delayed.
- Once the algorithm has decided on the reward, it needs to choose the action that should be performed in the current state.
- This is known as the **policy**.
- This is done based on some combination of exploration and exploitation (reinforcement learning is basically a search method), which in this case means deciding whether to take the action that gave the highest reward last time we were in this state, or trying out a different action in the hope of finding something even better.

EXAMPLE: GETTING LOST

- You arrive in a foreign city exhausted after many hours of flying, catch the train into town and stagger into a backpacker's hostel without noticing much of your surroundings.

- When you wake up it is dark and you are starving, so you set off for a wander around town looking for somewhere to eat.
- Unfortunately, it is 3 a.m. and, even more unfortunately, you soon realize that you are completely lost.
- To make matters worse, you discover that you can't remember the name of the backpacker's, or much else about it except that it is in one of the old squares.
- Of course, that doesn't help much because this part of the city pretty much consists of old squares.
- There are only two things in your favour: you are fairly sure that you'll recognise the building, and you've studied reinforcement learning and decide to apply it.
- You are sure that you've only walked through the old part of the city, so you don't need to worry about any street that doesn't.
- So at the next bus stop you come to, have a proper look at the map, and note down the map of the old town squares, which turns out to look like Figure 11.3.

- As you finish drawing the map you notice a 24-hour shop and buy as many bags of potato chips as you can fit into your pockets.
- As a reinforcement learner you decide to reward yourself when you take actions that lead to the backpacker's.
- you decide that the only one that will work is to eat until you can eat no more when you actually get to the backpacker's, and not to reward yourself at all until then.
- Hoping that you don't faint from hunger, Inspired by the idea of food, you decide that the backpacker's is almost definitely in the square labelled F on the map, because its name seems vaguely familiar.

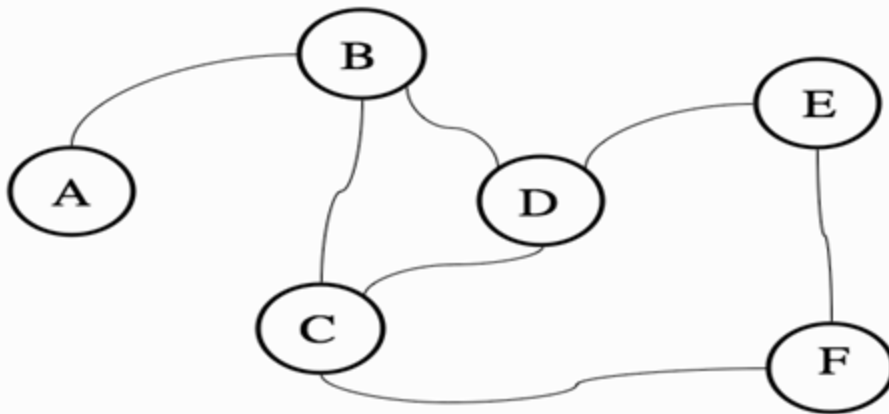
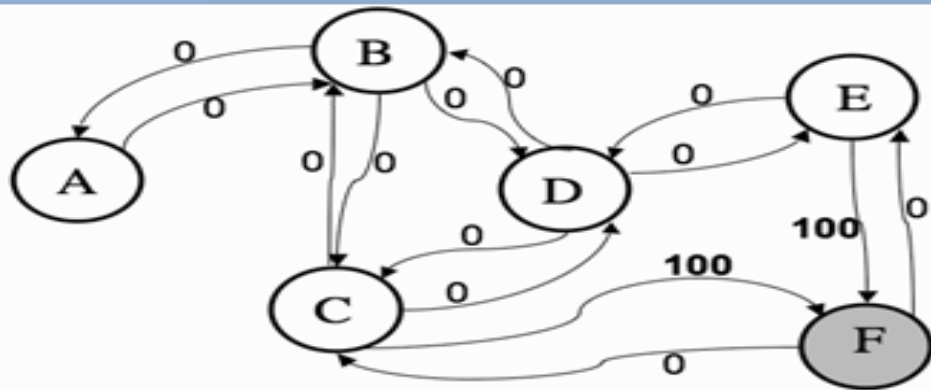


FIGURE 11.3 The old town that you find yourself lost in.



Current State	Next State					
	A	B	C	D	E	F
A	-5	0	-	-	-	-
B	0	-5	0	0	-	-
C	-	0	-5	0	-	100
D	-	0	0	-5	0	-
E	-	-	-	0	-5	100
F	-	-	0	-	0	-

FIGURE 11.4 The state diagram if you are correct and the backpacker's is in square (state) F. The connections from each state back into itself (meaning that you don't move) are not shown, to avoid the figure getting too complicated. They are each worth -5 (except for staying in state F, which means that you are in the backpacker's).

- This results in the reward matrix R shown below (where '-' shows that there is no link) and also in Figure 11.4.
- As a reinforcement learner you don't actually know the reward matrix.
- Assume that now reached a stage of tiredness where you can't even read what is on your paper properly.
- Having got this set up we've reached the stage where we need to do some learning, but for now we'll leave you stranded in that foreign city and flesh out a few of the things that we've talked about so far.

STATE AND ACTION SPACES

- Our reinforcement learner is basically a search algorithm, and obviously the larger the number of states that the algorithm has to search through, the longer it will take to find a good solution.
- The set of all states that are possible for the learner to experience is known as the state space.
- There is a corresponding action space that contains all of the possible actions.
- If we can reduce the size of the state space and action space, then it is almost always a good idea, providing that it does not oversimplify the problem.

CARROTS AND STICKS: THE REWARD FUNCTION

- The basic idea of the learner is that it will choose the action that gets the maximum expected reward.
- The reward function takes the current state and the chosen action and produces a numerical reward based on them.
- Ex: if we are in state A , and choose the action of doing nothing, so that we remain in state A , get a reward of -5 .
- Note that the reward can be positive or negative, with the latter corresponding to ‘punishment’, showing that particular actions should be avoided.
- It is therefore usually a bad idea to include sub-goals, because the learner can find methods of achieving the sub-goals without actually achieving the real goal.

DISCOUNTING

- The solution to this problem is known as discounting, take into account how certain we can be about things that happen in the future: there is lots of uncertainty in the learning anyway, so we should discount our predictions of rewards in the future according to how much chance there is that they are wrong.
- The rewards that we expect to get very soon are probably going to be more accurate predictions than those a long time in the future, because lots of other things might change.

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{k-1} r_k + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (11.1)$$

ACTION SELECTION

- Compute the average reward that has been received each time in the past.
- This is known as $Q_{s,t}(a)$, where s is the state, a is the action, and t is the number of times that the action has been taken before in this state.
- This will eventually converge to the true prediction of the reward for that action.
- Based on the current average reward predictions, there are three methods of choosing action a that are worth thinking about for reinforcement learning.
- The Methods are 1. **Greedy** 2. **ϵ -greedy** 3. **Soft-max**

POLICY

- Considered different action selection methods, such as ϵ -greedy and soft-max.
- The aim of the action selection is to trade off exploration and exploitation in such a way as to maximize the expected reward into the future.
- Make an explicit decision and always take the optimal choice at each stage, and not do exploration any more.
- Choice of which action to take in each state in order to get optimal results is known as the policy.

REFERENCES

- 1. Stephen Marsland, —Machine Learning – An Algorithmic Perspective, Second Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2014.